Practice Makes Operators

Anonymous Author(s)
Affiliation
Address
email

Abstract

Meta-RL agents adapt by acting, updating fast parameters, writing memory, invoking routines, and deciding when to halt adaptation. We study when repeated adaptation traces can be consolidated into reusable hierarchical operators. We introduce a certified compilation framework that operators can use to perform closed-loop stochastic computations over adaptation states, rank-decreasing calls and finite local-decision timeouts ensure executable recursion, primitive unrolling makes heterogeneous libraries comparable, and lower-confidence certificates require accepted edits to improve a resource-aware energy. The resulting recurrence threshold states that hierarchy is justified only when repeated savings exceed operator cost, value risk, and statistical uncertainty. This provides a theoretical foundation for meta-RL systems that grow executable operator libraries from recurring adaptation traces.

1 Introduction

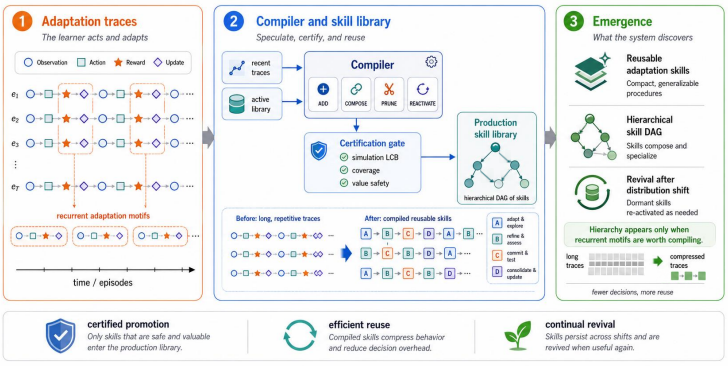

Figure 1: Overview of the framework.

Meta-reinforcement learning studies agents that improve their behavior from experience inside a task. Much of the field asks how to represent this online improvement. Gradient-based methods learn initial conditions for rapid parameter updates [6]. Latent-context methods infer task variables from interaction histories [15, 21]. Recurrent and sequence-model agents internalize adaptation inside hidden states or long contexts [3, 8, 12]. Recent language-agent and behavior-modeling work shows that exploration itself can be learned in context [10, 20]. These approaches make adaptation faster, yet the adaptation procedure usually remains implicit inside parameters, activations, or context.

In this paper we study a different unit of structure, summarized in Figure 1, which presents the theoretical core of an end-to-end meta-RL framework. During online adaptation, an agent generates traces that include environment actions, structured fast updates, memory writes, operator invocations, and adaptation-halting decisions. These traces can repeat across tasks. A recurring trace may express

a useful adaptation routine, such as probing a region, resetting a fast adapter, calibrating a local value estimate, or invoking a previously learned routine. We ask when such a recurring adaptation procedure can be compiled into a new closed-loop operator and added to a growing hierarchy.

Our question connects temporal abstraction with algorithm discovery. Options and SMDPs show how temporally extended decisions can be formalized in RL [1, 17, 19], while library and algorithm discovery systems show how generate–evaluate–retain loops can produce reusable computational objects [4, 7, 11, 13, 14]. Options abstract acting; operators abstract adapting. Appendix C formalizes options as a rank-zero no-update, no-memory projection of operators and contrasts gradient-trained options with statistically admitted operators. Test-time training adds a complementary signal that deployed models can update selected fast weights during inference [5, 16]. Appendix B discusses more related work.

2 Preliminary

A meta-RL adaptation trace records how the agent acts, updates its internal state, changes fast parameters, invokes previously learned routines, and decides when an adaptation episode is complete. Two agents may induce similar behavior trajectories while using different adaptation procedures.

2.1 Task families and adaptation state

Let tasks be indexed by a latent variable $\omega \sim p(\omega)$. We distinguish the latent environment state s_t from the observation o_t. Conditional on ω, a fully observed environment evolves as

$$s_{t+1} \sim P_\omega(\cdot \mid s_t, a_t), \qquad r_t \sim R_\omega(s_t, a_t),$$

with observations sampled from an observation kernel when the process is partially observed. In meta-RL, both s_t and ω may be hidden during adaptation. Therefore an observation alone is generally insufficient for Markov decision making. We use the belief-state form

$$x_t = (\mathfrak{b}_t, w_t, m_t, \nu_t) \in \mathcal{X}, \qquad \mathfrak{b}_t(s, \omega) = p(s_t = s, \omega \mid h_t).$$

Here w_t denotes fast weights or fast parameters, m_t is recurrent memory or optimizer state, and ν_t is execution state used by operator calls. In a fully observed MDP, this reduces to $x_t = (s_t, b_t, w_t, m_t, \nu_t)$ with $b_t(\omega) = p(\omega \mid h_t)$. In implementations, the belief may be approximated or represented implicitly by m_t.

Assumption 1 (Exact adaptation state)**.** *For the theoretical process, $\mathfrak{b}_t$ is the following exact posterior over latent environment state and task:*

$$\mathfrak{b}_t(s, \omega) = p(s_t = s, \omega \mid h_t),$$

where h_t is the complete adaptation history available before choosing the next primitive decision.

This exact-posterior state Markovizes the adaptation process for any fixed library and controller; the formal statement is given in Proposition 1. If b_t is omitted, the same construction should be read as a partially observed process controlled by a recurrent state.

2.2 Primitive adaptation decisions

The primitive decisions of the adaptation process are

$$\mathcal{U}_0 = \mathcal{A}_{\text{env}} \cup \mathcal{U}_{\text{upd}} \cup \mathcal{U}_{\text{mem}} \cup \{\text{halt}_{\text{adapt}}\}.$$

The primitive action $\text{halt}_{\text{adapt}}$ terminates the adaptation episode. It is distinct from the return bit used by an active operator to hand control back to its caller. The set $\mathcal{U}_{\text{upd}}$ is a structured family of update operators, giving the primitive interface a controlled update vocabulary. A primitive update is written

$$\delta_t = U_{\alpha_t}(x_t, \zeta_t), \qquad \alpha_t \in \mathcal{U}_{\text{upd}},$$

where ζ_t is a bounded argument such as a step size, adapter block, rank, loss head, memory slot, or residual coefficient. This restriction is deliberate. If the primitive update action were an arbitrary vector in $\mathbb{R}^d$, the formalism would become vacuously expressive, since discovering an adaptation operator would require searching over the entire fast-weight space. The structured update family makes the adaptation process an object that can be logged, compressed, and evaluated. Appendix D.4 explains why this restriction gives operator compilation nontrivial semantic content.

A primitive adaptation trace is

$$\tau_0 = (x_0, u_0, c_0, x_1, u_1, c_1, \ldots, x_T), \qquad u_t \in \mathcal{U}_0,$$

where c_t is a bounded adaptation cost. The cost may include environment interaction, update computation, memory writes, and adaptation-halt penalties. We assume finite horizons in the main paper and give the discounted extension in Appendix D.

2.3 Semi-Markov execution

Now we introduce learned operators that run for multiple primitive steps. The right mathematical foundation is therefore a semi-Markov decision process. We use local-decision timeout semantics. An operator's timeout H_κ counts only decisions made by κ itself and excludes primitive time spent inside lower-rank subcalls. After the jth local decision of κ returns control to that operator frame, execution samples an operator-return bit $\sigma_{\kappa,j} \sim \mathrm{Bernoulli}(\beta_\kappa(x_{\kappa,j}))$, and the operator returns to its caller when $\sigma_{\kappa,j} = 1$ or $j = H_\kappa$. Its primitive wall-clock duration τ_κ includes all primitive decisions made inside lower-rank subcalls.

Invoking κ from state x integrates over the intra-operator decisions, primitive randomness, lower-rank calls, and return bits to induce a kernel $K_\kappa(dx', d\ell, dg \mid x)$, where x' is the terminal adaptation state, ℓ is the duration, and g is the accumulated primitive reward or cost. This is the same role options play in hierarchical RL, except that the underlying state is an adaptation state and the operator may execute environment actions, structured updates, memory operations, and lower-rank calls.

Definition 1 (Adaptation SMDP with a fixed library)**.** *For a fixed operator library $\mathcal{L}$ and fixed orchestration controller π, the adaptation process is the semi-Markov process*

$$\mathcal{M}(\mathcal{L}, \pi) = (\mathcal{X}, \mathcal{U}_\mathcal{L}, P_\mathcal{L}, C_\mathcal{L}, \gamma),$$

where $\mathcal{U}_\mathcal{L} = \mathcal{U}_0 \cup \{\mathrm{invoke}(\kappa) : \kappa \in V_\mathcal{L}\}$.

The word "fixed" is important. Once a compiler edits $\mathcal{L}$, the action set and induced transition kernels change. The full training process is therefore a sequence of SMDPs, with each fixed library inducing its own stationary object. Sections 3 and 4 describe the fixed object that a compiler may safely edit.

3 Hierarchical Adaptation Operators

We now define nonlinear closed-loop stochastic operators that can replace repeated adaptation traces. An operator may be implemented by a recurrent network, a transformer with state, a small controller over fast adapters, or any other measurable transducer. Its mathematical identity is the kernel it induces over adaptation states and primitive unrollings.

3.1 Closed-loop adaptation operators

Definition 2 (Adaptation operator)**.** *An adaptation operator is a tuple*

$$\kappa = (I_\kappa, \pi_\kappa, \beta_\kappa, H_\kappa, r_\kappa, \theta_\kappa).$$

Here $I_\kappa \subseteq \mathcal{X}$ is an initiation set, π_κ is an intra-operator policy, $\beta_\kappa : \mathcal{X} \to [0,1]$ is an operator-return probability used to sample a Bernoulli return bit during execution, $H_\kappa < \infty$ is a local-decision timeout, $r_\kappa \in \mathbb{N}$ is a rank, and θ_κ are parameters.

A library is a directed graph $\mathcal{L} = (V, E, \Theta)$, where V is a finite set of operators and $E \subseteq V \times V$ records admissible calls. If $(\kappa, \kappa') \in E$, then π_κ may choose $\mathrm{invoke}(\kappa')$ as one of its internal decisions. Only executable edges have theoretical content.

The intra-operator policy acts over

$$\mathcal{U}_\kappa(x) = \mathcal{U}_0 \cup \{\mathrm{invoke}(\kappa') : (\kappa, \kappa') \in E,\ x \in I_{\kappa'}\}.$$

This functional view of composition lets a higher-rank operator call lower-rank operators as part of its own closed-loop computation.

3.2 Rank-constrained recursion

If operators can call each other freely, execution may fail to terminate and the induced kernel may be undefined. We rule this out by requiring calls to decrease rank.

Assumption 2 (Rank-decreasing calls)**.** *For every edge $(\kappa, \kappa') \in E$, $r_{\kappa'} < r_\kappa$. Every operator has finite local-decision timeout $H_\kappa < \infty$.*

Theorem 1 (Well-founded recursive execution)**.** *Under Assumption 2, every operator invocation terminates after finitely many primitive decisions.*

The theorem is simple and supplies the execution invariant that makes recursive operators legitimate. For each fixed finite library, let $T_{\max}$ denote the resulting maximum primitive wall-clock length of any admissible unrolled invocation. The DAG provides the scaffold for executable hierarchy; certified recurrence explains why a useful hierarchy should be introduced. Once the compiler introduces a hierarchy, the DAG ensures well-defined execution.

3.3 Primitive unrolling

A learned operator is nonlinear and may be hard to interpret. We therefore avoid giving it symbolic semantics. Instead, we give it operational semantics by unrolling its execution into primitive adaptation traces. Let $\mathcal{T}_0$ denote the set of finite primitive adaptation traces over $\mathcal{U}_0$.

Definition 3 (Primitive unrolling kernel)**.** *For a library $\mathcal{L}$ satisfying Assumption 2, the primitive unrolling kernel of operator κ is*

$$U_{\mathcal{L}}(\cdot \mid x, \kappa) \in \Delta(\mathcal{T}_0),$$

the distribution over primitive adaptation traces generated by executing κ from state x and recursively replacing every lower-rank invocation by its own primitive execution.

Theorem 2 (Existence of primitive unrolling)**.** *Under Assumption 2, $U_{\mathcal{L}}(\cdot \mid x, \kappa)$ is well-defined for every $x \in I_\kappa$ and every $\kappa \in V$.*

This theorem is the key device that lets us discuss hierarchy through operational behavior. A rank-two operator only needs to induce a finite distribution over primitive act-update-memory traces. The primitive unrolling kernel is also what lets us compare libraries with different operator sets as all high-level executions can be projected back to the same primitive trace space.

4 Certified Compilation of Adaptation Traces

We now define what it means to compile traces into operators. The compiler certifies local improvement under a fixed evaluation protocol, which gives a tractable and auditable criterion for admitting a proposed operator.

4.1 Evaluated objects

Performance is determined by the library together with the controller that decides when to invoke its operators. We therefore evaluate either a pair $(\mathcal{L}, \pi)$ or a library together with a fixed orchestration-training rule. In the main text we use the pair notation.

Let $\bar{P}^{\pi}_{\mathcal{L}}$ be the distribution over primitive adaptation traces induced by running π in $\mathcal{M}(\mathcal{L}, \pi)$ and unrolling every operator invocation through Definition 3. This distribution lives on $\mathcal{T}_0$ even though π may use high-level operators. Thus two libraries with different operator sets remain comparable through the same primitive trace space.

Definition 4 (Resource-aware energy)**.** *For a fixed controller π, define*

$$\mathcal{E}_{\text{deploy}}(\mathcal{L}, \pi) = L_{\text{lib}}(\mathcal{L}) + L_{\text{ctrl}}(\pi \mid \mathcal{L}) + \mathbb{E}_{\tau \sim \bar{P}^{\pi}_{\mathcal{L}}}[C(\tau)] + \lambda[J_{\min} - J(\mathcal{L}, \pi)]_+.$$

Here $L_{\text{lib}}(\mathcal{L})$ is the library description cost, $L_{\text{ctrl}}(\pi \mid \mathcal{L})$ prices the controller relative to the action space induced by $\mathcal{L}$, $C(\tau)$ is primitive adaptation cost, and J is expected task return.

The hinge term prevents degenerate compression. A library that shortens traces while destroying return fails this acceptance criterion. The controller term is necessary because adding operators changes the effective action space; with this term, benefits from increased controller capacity are separated from operator benefits. In the main theory the library cost is fixed before certification and decomposes as

$$L_{\text{lib}}(\mathcal{L}) = L_{\text{graph}}(\mathcal{L}) + L_{\text{rank}}(\mathcal{L}) + L_{\text{arch}}(\mathcal{L}) + L_Q(\theta_{\mathcal{L}}) + L_{\text{call}}(\mathcal{L}).$$

If this is implemented as a weighted surrogate on vertices, edges, ranks, architectures, quantized parameters, and call interfaces, the weights and quantization rule are part of the certification protocol and must be fixed before validation. For an evaluated pair $A = (\mathcal{L}_A, \pi_A)$, write

$$L_{\text{struct}}(A) = L_{\text{lib}}(\mathcal{L}_A) + L_{\text{ctrl}}(\pi_A \mid \mathcal{L}_A).$$

We use three objectives. $\mathcal{E}_{\text{deploy}}(\mathcal{L}, \pi)$ is the full closed-loop deployment energy above. Given aligned validation occurrences $\mathcal{I}_\kappa^{\text{val}} = \{1, \dots, n\}$, define the validation objective by

$$\mathcal{E}_{\text{val}}^{\mathcal{I}_\kappa^{\text{val}}}(\kappa; \mathcal{L}) := L_{\text{add}}(\kappa; \mathcal{L}) - \sum_{i=1}^{n} \mathbb{E}[D_i(\kappa)].$$

The base decision not to add κ has validation objective 0, so negative value means certified validation improvement. The structural charge L_{add} is fixed by the validation protocol: frozen-controller recurrence sets the controller increment to zero, while matched-controller validation must include the corresponding L_{ctrl} increment. The recurrence theorem certifies only $\mathcal{E}_{\text{val}}^{\mathcal{I}_\kappa^{\text{val}}}$; deployment improvement follows only after composition with the local-to-global conditions in Theorem 5 or with the direct certificate in Theorem 3.

Cross-library comparability. The unrolling construction gives a stable semantic base. Adding an operator changes the high-level action set, so directly comparing high-level traces before and after compilation is ill-posed. Comparing their primitive unrollings is well-posed; the elementary total-variation bound is stated as Proposition 2 in Appendix F.3.

4.2 Candidate edits

A compiler proposes a local edit $o : \mathcal{L} \mapsto o(\mathcal{L})$. We use two edit types: $\text{ADD}(\kappa)$, $\text{COMPOSE}(\kappa; S)$, where S is a set of lower-rank operators that the new operator may invoke. Pruning and revival can be added later, while ADD and COMPOSE suffice to define certified compilation.

The true improvement of an edit is

$$\Delta\mathcal{E}_{\text{deploy}}(o; \pi, \pi') = \mathcal{E}_{\text{deploy}}(\mathcal{L}, \pi) - \mathcal{E}_{\text{deploy}}(o(\mathcal{L}), \pi').$$

The controller choice must be fixed by protocol. One may freeze the controller and set $\pi' = \pi$, or allow both sides a matched adaptation budget under a specified training rule. This convention separates operator improvement from gains due to extra controller training.

4.3 Local certified improvement

The compiler accepts an edit only if a conservative certificate lower-bounds the energy improvement. Let $h(J) = [J_{\min} - J]_+$. For a proposed edit taking $A = (\mathcal{L}, \pi)$ to $B = (o(\mathcal{L}), \pi')$, assume estimates $\hat{R}_A, \hat{R}_B, \hat{J}_A, \hat{J}_B$ with radii $b_R(A), b_R(B), b_J(A), b_J(B)$ for resource cost and return. Since h is decreasing, the conservative lower bound is

$$\begin{aligned}\text{LCB}(\Delta\mathcal{E}_{\text{deploy}}) = [L_{\text{struct}}(A) - L_{\text{struct}}(B)] + [\hat{R}_A - \hat{R}_B - b_R(A) - b_R(B)] \\ + \lambda \left[h(\hat{J}_A + b_J(A)) - h(\hat{J}_B - b_J(B)) \right].\end{aligned}$$

Theorem 3 (Certified local improvement)**.** *If all component confidence intervals are simultaneously valid and* $\text{LCB}(\Delta\mathcal{E}_{\text{deploy}}) > 0$*, then the true energy improvement satisfies* $\Delta\mathcal{E}_{\text{deploy}} > 0$.

This certification result says that an accepted edit improves the specified energy under the specified evaluation protocol. It leaves optimal operator search, hierarchy emergence, and replay validity under support conditions to later assumptions.

5 Emergent Hierarchy

Sections 3 and 4 define how a candidate operator can be executed and how a proposed edit can be certified. They leave the source of hierarchy to the recurrence argument below. A DAG can be imposed by hand. A useful hierarchy has to earn its place by reducing the cost of future adaptation.

The central observation is that recurrence gives hierarchy its value. A single unusual trace can always be memorized, while compression requires reuse across traces. A reusable operator becomes valuable when its one-time description cost is amortized across many future executions. This gives a concrete account of emergence, in which a higher-level operator appears when recurrence-weighted savings exceed operator cost, value risk, and statistical uncertainty.

5.1 Operational motifs

We use the word *motif* for an empirically recurring adaptation pattern. A motif may be symbolic, human-readable, or a family of nonlinear trajectories through fast weights, memory states, update choices, and environment actions. Formally, a proposed operator κ is paired with an alignment rule $\mathcal{A}_\kappa : \mathcal{T}_0 \to 2^{\mathcal{I}}$, which identifies candidate occurrences in primitive validation traces; the held-out occurrence set used for certification is denoted $\mathcal{I}_\kappa^{\mathrm{val}}$. The rule may be produced by clustering, segmentation, a proposal model, a hand-specified template, or any other mining procedure. Certification treats this rule as part of the candidate. This choice avoids circularity by tying usefulness to a fixed counterfactual validation protocol. A mined motif becomes useful only when replacing held-out occurrences by κ reduces the specified energy surrogate.

For each held-out occurrence i, define a random saving by a fixed isolated counterfactual experiment, which starts the base segment and the operator segment from the same logged adaptation state, uses the same prescribed continuation or rollout budget, and measures a complete local energy difference

$$D_i(\kappa) = \Phi_i(\mathrm{base}) - \Phi_i(\mathrm{op}),$$

where Φ_i includes primitive adaptation cost, call overhead, primitive unrolling, and a local value-risk term. A typical choice is

$$\Phi_i(\tau_{i:i+\ell}) = C_i(\tau_{i:i+\ell}) + \lambda[V_{\min}(x_i) - \widehat{V}(x_{i+\ell})]_+ + C_{\mathrm{call}}(\tau_{i:i+\ell}),$$

with calibrated lower-confidence control for $\widehat{V}$ under the validation distribution. Thus D_i is treated as an isolated validation estimand tied to the controlled certificate protocol. When invoking the local-to-global theorem below, the segment-only component is denoted D_i^{seg}; augmented local objectives using $\widehat{V}$ require an explicit residual approximation bound.

The validation protocol must also specify how the counterfactual is identified. It may use a resettable generative model over adaptation states, or off-policy replay with support over the full hybrid primitive

$$z_t = (a_t, \alpha_t, \zeta_t, \eta_t, i_t, \sigma_t),$$

containing the environment action, update choice and argument, memory choice, invocation choice, and operator-return bit. Logs containing only (o_t, a_t, r_t, o_{t+1}) are insufficient for certifying operators that change updates, memory writes, invocations, or operator-return decisions.

The operator has a one-time incremental library cost

$$L_{\mathrm{add}}^{\mathrm{lib}}(\kappa; \mathcal{L}) = L_{\mathrm{lib}}(\mathcal{L} \cup \{\kappa\}) - L_{\mathrm{lib}}(\mathcal{L}),$$

possibly including new edges and parameters. The recurrence threshold charges the protocol-specific structural increment

$$L_{\mathrm{add}}(\kappa; \mathcal{L}) = L_{\mathrm{add}}^{\mathrm{lib}}(\kappa; \mathcal{L}) + \Delta L_{\mathrm{ctrl}}^{\mathrm{prot}}(\kappa; \mathcal{L}),$$

where $\Delta L_{\mathrm{ctrl}}^{\mathrm{prot}} = 0$ for frozen-controller replacement and is the matched-controller increment otherwise. A proposed operator earns acceptance through a conservative aggregate certificate supported by repeated held-out evidence. Let $u_n(\delta)$ be a valid lower-confidence penalty for the cumulative saving on n held-out occurrences.

Let $\bar{S}_n(\kappa) = \sum_{i=1}^n \mathbb{E}[D_i(\kappa)]$ denote expected validation saving under this fixed sampling protocol. The penalty $u_n(\delta)$ may come from independent regenerated episodes, a martingale confidence sequence for adaptively checked occurrences, or a mixing/blocking bound for trajectory-correlated occurrences. The recurrence theorem assumes only that the chosen penalty is valid for the actual sampling process.

Theorem 4 (Recurrence threshold for operator introduction)**.** *Suppose a candidate operator κ has n held-out aligned occurrences and bounded isolated counterfactual savings $D_1(\kappa), \ldots, D_n(\kappa)$ under a fixed validation protocol with counterfactual support as specified above. If the penalty $u_n(\delta)$ is valid for the occurrence-sampling process, so that with probability at least $1-\delta$, $\bar{S}_n(\kappa) \geq \sum_{i=1}^n \widehat{D}_i(\kappa) - u_n(\delta)$, then, with the same probability,*

$$\sum_{i=1}^n \widehat{D}_i(\kappa) > L_{\mathrm{add}}(\kappa; \mathcal{L}) + u_n(\delta) \quad \Longrightarrow \quad \mathcal{E}_{\mathrm{val}}^{\mathcal{I}_\kappa^{\mathrm{val}}}(\kappa; \mathcal{L}) < 0.$$

The theorem certifies the fixed isolated validation protocol, giving a local guarantee whose deployment relevance must be established separately. Replacing a segment can change terminal states, future controller actions, future motif occurrences, and return. The statement therefore uses the fixed isolated validation protocol that defines D_i and $\bar{S}_n$. Under that protocol, the evidence required for introduction scales intuitively because costly or noisy operators require more recurrence, while cheap operators that replace frequent routines can enter early.

To clarify, the hierarchy emerges through the acceptance rule and its stated validation protocol. Recurring patterns enter the hierarchy exactly when they beat their cost and uncertainty. Rejections therefore provide meaningful evidence about the absence of certified reusable structure.

Definition 5 (Certified hierarchy-emergence)**.** *Let $(\mathcal{L}_t, \pi_t)$ be the compiler trajectory, ξ_t be the validation start-state distribution at compiler step t, and $\mathrm{LCB}_t(\Delta\mathcal{E}_{\mathrm{val}}(\kappa))$ denote the lower confidence bound used by the recurrence certificate. For rank r and invocation threshold $\eta > 0$, define*

$$\mathcal{H}_t(r,\eta) = \left\{ \begin{array}{l} \exists \kappa \in \mathcal{L}_{t+1} \setminus \mathcal{L}_t : \; r_\kappa = r, \quad \Pr_{x\sim\xi_t}[\kappa \text{ invokes some } \kappa' \text{ with } r_{\kappa'} < r \mid x] \geq \eta, \\ \mathrm{LCB}_t(\Delta\mathcal{E}_{\mathrm{val}}(\kappa)) > 0 \end{array} \right\}.$$

Thus emergent hierarchy requires a high-rank operator to call lower-rank operators with sufficient probability; otherwise the operator is merely a high-rank syntactic node.

The immediate no-free-hierarchy consequence is deferred to Corollary 3: operators that do not beat structural cost plus statistical uncertainty are not certified.

5.2 Recursive compression

The first accepted operators compress primitive traces. Later operators may compress traces that already contain lower-rank invocations. This is the mechanism that turns isolated operator introduction into hierarchy. Let $\mathcal{L}$ already contain operators of ranks below r. A rank-r candidate may execute primitive decisions and invoke lower-rank operators. Its savings are still evaluated through primitive unrolling. This primitive evaluation is crucial because a rank-r operator earns credit through its primitive consequences after every high-level call is expanded. Its call sequence is expanded back to primitive adaptation traces before costs and returns are compared. The resulting algebraic compression condition is stated as Corollary 4 in the appendix.

Theorem 5 (Drift-free local-to-global deployment lower bound)**.** *Consider a frozen-controller deployment comparison replacing a fixed list of n base-trajectory occurrences by κ. The listed intervals are either pairwise non-overlapping in primitive time, or a deterministic replacement schedule is fixed in advance and resolves overlaps into scheduled marginal replacements. All quantities below are defined after this scheduling, so no primitive cost or continuation state is charged to more than one occurrence. Let $D_i^{\mathrm{seg}}(\kappa)$ be the segment-only saving, excluding continuation-value proxies, and define $\bar{S}_n^{\mathrm{seg}}(\kappa) = \sum_{i=1}^n \mathbb{E}[D_i^{\mathrm{seg}}(\kappa)]$. For occurrence i, let $P^{\mathrm{base}}_{\mathrm{term},i}(\cdot \mid x_i)$ and $P^{\kappa}_{\mathrm{term},i}(\cdot \mid x_i)$ be the terminal-state laws of the base segment and operator segment. Assume a regenerative decomposition of the full deployment-energy improvement: there exist measurable continuation-energy functions G_i with $|G_i(x)| \leq V_{\max}$ such that*

$$\Delta\mathcal{E}_{\mathrm{deploy}} = \bar{S}_n^{\mathrm{seg}}(\kappa) - L_{\mathrm{add}}(\kappa;\mathcal{L}) - \sum_{i=1}^n \left[\mathbb{E}_{x\sim P^{\kappa}_{\mathrm{term},i}} G_i(x) - \mathbb{E}_{x\sim P^{\mathrm{base}}_{\mathrm{term},i}} G_i(x) \right].$$

Here G_i is part of the evaluation protocol and includes all downstream primitive costs and return-penalty effects, including the global hinge contribution. Suppose also that

$$\mathrm{TV}\big(P^{\mathrm{base}}_{\mathrm{term},i}(\cdot \mid x_i), P^{\kappa}_{\mathrm{term},i}(\cdot \mid x_i)\big) \leq \epsilon_i$$

for each occurrence. Then the deployment-energy improvement obeys

$$\Delta\mathcal{E}_{\mathrm{deploy}} \geq \bar{S}_n^{\mathrm{seg}}(\kappa) - L_{\mathrm{add}}(\kappa;\mathcal{L}) - 2V_{\max} \sum_{i=1}^n \epsilon_i.$$

In the regeneration case $\epsilon_i = 0$, scheduled segment savings are additive. The regenerative decomposition and non-overlap or scheduling condition are substantive assumptions: the hinge penalty in $\mathcal{E}_{\mathrm{deploy}}$ is global and nonlinear, and overlapping intervals can otherwise double-count primitive costs or continuation states. Without these conditions, comparisons with retrained controllers, replacement-induced changes in future occurrence frequencies, overlapping motifs, or non-regenerative hinge effects require a separate deployment certificate.

The theorem stack is intentionally separated: rank decrease and local-decision timeouts give finite execution, recurrence lower confidence gives $\mathcal{E}_{\mathrm{val}}^{\mathcal{I}_\kappa^{\mathrm{val}}} < 0$, and coupling, regeneration, or direct deployment certification is required for deployment improvement.

5.3 What is discovered

Under this view, the discovered object is an adaptation operator with operational semantics. Its role is operational. It maps an adaptation state to a finite stochastic execution that may act, update, write memory, call lower-rank operators, and return to its caller. The DAG records which calls are executable. Certification determines which nodes and edges are admitted. Recurrence determines when the cost of adding the node is justified.

Theorem 6 (Emergence sandwich)**.** *Fix T and confidence levels $(\delta_t)_{t=1}^T$ with $\sum_t \delta_t \le \delta$. Suppose each proposed edit is checked for rank, primitive unrolling, counterfactual support, recurrence lower confidence, and either frozen-validation or deployment certification. Define*

$$\Delta\mathcal{E}_{\mathrm{val},t}^{\mathrm{loc}} := -\mathcal{E}_{\mathrm{val}}^{\mathcal{I}_{\kappa_t}^{\mathrm{val}}}(\kappa_t; \mathcal{L}_t) = \sum_{i=1}^{n_t} \mathbb{E}[D_{t,i}] - L_{\mathrm{add}}(\kappa_t; \mathcal{L}_t).$$

When the local-to-global coupling layer is invoked, also define the segment-only validation gain

$$\Delta\mathcal{E}_{\mathrm{val},t}^{\mathrm{seg}} := \sum_{i=1}^{n_t} \mathbb{E}[D_{t,i}^{\mathrm{seg}}] - L_{\mathrm{add}}(\kappa_t; \mathcal{L}_t),$$

where $D_{t,i}^{\mathrm{seg}}$ excludes continuation-value proxies. With probability at least $1-\delta$, every accepted edit κ_t satisfies well-founded execution and

$$\sum_{i=1}^{n_t} \widehat{D}_{t,i} > L_{\mathrm{add}}(\kappa_t; \mathcal{L}_t) + u_{n_t}(\delta_t), \qquad \Delta\mathcal{E}_{\mathrm{val},t}^{\mathrm{loc}} > 0.$$

If the coupling in Theorem 5 also hold at step t with segment-only validation savings, then

$$\Delta\mathcal{E}_{\mathrm{deploy},t} \ge \Delta\mathcal{E}_{\mathrm{val},t}^{\mathrm{seg}} - 2V_{\max} \sum_{i=1}^{n_t} \epsilon_{t,i}.$$

This theorem layers guarantees with clear roles at each level. The lower layer is execution safety, the middle layer is recurrence-certified local validation improvement, and the upper layer is deployment relevance only under explicit segment-only coupling or direct deployment certification. Proposal support remains a search-completeness condition; certification supplies the premise that hierarchy is useful.

6 Proposal Support and Revival

Certification controls what enters the library, while proposal support governs whether useful operators are proposed. This distinction is important. A perfect certificate can reject bad edits and still miss every useful one if the proposal process lacks support. We therefore treat discovery as a separate stochastic search problem.

Let Q_t be the proposal distribution over candidate edits at compiler step t. A practical compiler may learn Q_t from previously accepted and rejected edits. Such learning can improve search efficiency while also risking collapse onto familiar operator types that stop proposing rare ones. We use a small revival component to prevent proposal extinction.

6.1 Type-level proposal floor

Let r denote a proposal type, such as adding a primitive-to-operator abstraction, composing lower-rank operators, reactivating a cached operator, or pruning an operator. We define

$$q_t(r \mid c_t) = (1-\rho_t)\tilde{q}_t(r \mid c_t) + \rho_t G_0(r), \qquad \rho_t \ge \rho_{\min} > 0,$$

where c_t is compiler context, $\tilde{q}_t$ is the learned proposal model, and G_0 is a fixed base distribution over proposal types.

The mixture gives the non-extinction bound in Proposition 6. Its interpretation is type-level: a proposal family remains reachable, while exact revival of a learned nonlinear operator requires either a stored copy or atomic proposal mass at old parameters.

6.2 Search-time implication

Let $\mathcal{G}$ be a useful candidate region. For example, $\mathcal{G}$ may contain all edits whose true energy improvement is positive and whose certificate has enough power to accept them. Suppose a proposal type $r^\star$ has base mass, and conditional on drawing that type the proposal lands in $\mathcal{G}$ with probability at least $p_\star$. If a candidate in $\mathcal{G}$ is accepted with probability at least $a_\star$, then the revival floor gives a finite hitting-time bound.

The precise geometric hitting bound is Theorem 8 in the appendix. It identifies the assumption needed for discovery claims, namely that useful operator neighborhoods have positive proposal mass. Appendix H.3 gives a finite-cover version showing how such mass, together with a positive recurrence margin and fresh validation samples, yields a high-probability discovery bound. If proposal support fails, no certification theorem can recover the missing candidate.

Dormant operators A cache changes the situation. Suppose a previously accepted operator is removed from the active DAG and stored in a dormant set $\mathcal{H}$. Reactivation can then propose the exact stored parameters. This makes revival qualitatively different from rediscovery. Type-level revival keeps a family of proposals alive; cache-level revival can resurrect a specific nonlinear operator. Revival preserves proposal support, and a dormant cache can reduce the cost of reactivating operators whose utility returns under distribution shift.

7 Discussion

In this paper, we present the theoretical core of an *emergen meta-RL framework* which builds agents that adapt online, record their own adaptation traces, discover recurring adaptation procedures, compile those procedures into reusable operators, and reuse them through a growing hierarchy. Here we focus on answering when it is meaningful to turn a recurring adaptation trace into a new operator, and when it is safe to add that operator to the agent's library.

The main contribution is to turn hierarchy in meta-RL from an architectural choice into a certifiable compilation step. A recurring adaptation pattern becomes a candidate operator only through its primitive consequences: after unrolling, it must save resource-aware cost, preserve value under the validation protocol, and pay for its own description and call overhead. The recurrence threshold gives the admission rule, while the emergence sandwich separates what is guaranteed locally, what is executable by construction, and what requires an explicit deployment certificate. In this sense, a higher-level operator earns its place in the library through repeated evidence.

The framework also separates the three ingredients needed for such emergence. Rank-decreasing calls and finite local-decision timeouts make recursive execution well-defined. Primitive unrolling gives a common trace space for comparing heterogeneous libraries before and after compilation. Lower-confidence certification supplies the statistical gate that prevents a few favorable traces from becoming new structure. Proposal floors and caches then state the complementary search condition: useful operator neighborhoods must remain reachable. Together, these results define a concrete compiler interface for future systems that learn adaptive policies, mine their traces, and grow executable operator libraries from recurring adaptation routines.

8 Conclusion

We studied when repeated meta-RL adaptation procedures can be consolidated into reusable hierarchical operators. We formalized operators as closed-loop stochastic computations over adaptation states and introduced a certified compilation framework in which rank-decreasing calls ensure finite recursive execution, primitive unrolling makes libraries comparable, and lower-confidence certificates require accepted edits to improve a resource-aware energy. The resulting recurrence threshold states that hierarchy is justified only when repeated savings exceed operator cost, value risk, and statistical uncertainty.

Our work provides a theoretical foundation for building emergent meta-RL systems that grow executable operator libraries from recurring adaptation traces and turns recurring adaptation patterns into reusable routines. When an agent repeatedly solves the same kind of adaptation problem, such as exploring a new maze, recovering after a task shift, or calibrating a new object, the system can package that behavior into an operator that can be called later.

References

[1] Bacon, P.-L., Harb, J., and Precup, D. The option-critic architecture. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.

[2] Dai, B., Nachum, O., Chow, Y., Li, L., Szepesvári, C., and Schuurmans, D. CoinDICE: Off-policy confidence interval estimation. In *Advances in Neural Information Processing Systems*, volume 33, pp. 9398–9411, 2020.

[3] Duan, Y., Schulman, J., Chen, X., Bartlett, P. L., Sutskever, I., and Abbeel, P. RL^2: Fast reinforcement learning via slow reinforcement learning. In *International Conference on Learning Representations*, 2017.

[4] Fawzi, A., Balog, M., Huang, A., Hubert, T., Romera-Paredes, B., Barekatain, M., Novikov, A., R. Ruiz, F. J., Schrittwieser, J., Swirszcz, G., et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022.

[5] Feng, G., Luo, S., Hua, K., Zhang, G., Huang, W., He, D., and Cai, T. In-place test-time training. In *International Conference on Learning Representations*, 2026.

[6] Finn, C., Abbeel, P., and Levine, S. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1126–1135. PMLR, 2017.

[7] Grand, G., Wong, L., Bowers, M., Olausson, T. X., Liu, M., Tenenbaum, J. B., and Andreas, J. LILO: Learning interpretable libraries by compressing and documenting code. In *International Conference on Learning Representations*, 2024.

[8] Grigsby, J., Fan, L., and Zhu, Y. AMAGO: Scalable in-context reinforcement learning for adaptive agents. In *International Conference on Learning Representations*, 2024.

[9] Howard, S. R., Ramdas, A., McAuliffe, J., and Sekhon, J. Time-uniform, nonparametric, nonasymptotic confidence sequences. *The Annals of Statistics*, 49(2):1055–1080, 2021. doi: 10.1214/20-AOS1991.

[10] Jiang, Y., Jiang, L., Teney, D., Moor, M., and Brbic, M. Meta-RL induces exploration in language agents. In *International Conference on Learning Representations*, 2026.

[11] Mankowitz, D. J., Michi, A., Zhernov, A., Gelmi, M., Selvi, M., Paduraru, C., Leurent, E., Iqbal, S., Lespiau, J.-B., Ahern, A., Koppe, T., Millikin, K., Gaffney, S., Elster, S., Broshear, J., Gamble, C., Milan, K., Tung, R., Hwang, M., Cemgil, T., Barekatain, M., Li, Y., Mandhane, A., Hubert, T., Schrittwieser, J., Hassabis, D., Kohli, P., Riedmiller, M., Vinyals, O., and Silver, D. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618:257–263, 2023. doi: 10.1038/s41586-023-06004-9.

[12] Mishra, N., Rohaninejad, M., Chen, X., and Abbeel, P. A simple neural attentive meta-learner. In *International Conference on Learning Representations*, 2018.

[13] Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J., Mehrabian, A., et al. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025.

[14] Oh, J., Farquhar, G., Kemaev, I., Calian, D. A., Hessel, M., Zintgraf, L., Singh, S., Van Hasselt, H., and Silver, D. Discovering state-of-the-art reinforcement learning algorithms. *Nature*, 648 (8093):312–319, 2025.

[15] Rakelly, K., Zhou, A., Finn, C., Levine, S., and Quillen, D. Efficient off-policy meta-reinforcement learning via probabilistic context variables. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 5331–5340. PMLR, 2019.

[16] Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A. A., and Hardt, M. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 9229–9248. PMLR, 2020.

[17] Sutton, R. S., Precup, D., and Singh, S. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1.

[18] Thomas, P., Theocharous, G., and Ghavamzadeh, M. High-confidence off-policy evaluation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015. doi: 10.1609/aaai.v29i1.9541.

[19] Vezhnevets, A. S., Osindero, S., Schaul, T., Heess, N., Jaderberg, M., Silver, D., and Kavukcuoglu, K. Feudal networks for hierarchical reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 3540–3549. PMLR, 2017.

[20] Wagenmaker, A., Zhou, Z., and Levine, S. Behavioral exploration: Learning to explore via in-context adaptation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*. PMLR, 2025.

[21] Zintgraf, L. M., Shiarlis, K., Igl, M., Schulze, S., Gal, Y., Hofmann, K., and Whiteson, S. VariBAD: A very good method for bayes-adaptive deep RL via meta-learning. In *International Conference on Learning Representations*, 2020.

Appendix

A World-Feedback Perspective

The framework can be read as a theory of how an agent converts feedback from its own interaction process into reusable adaptation structure. The feedback used by the compiler is a structured collection of signals generated throughout adaptation. It is a collection of measurable signals generated while the agent adapts, including task return, interaction count, update cost, memory cost, call overhead, latency, constraint violations, and other quantities that enter the primitive trace cost

$$C(\tau) = \sum_t c(x_t, u_t, x_{t+1}).$$

These signals are world-grounded in the sense that they arise from the coupled system formed by the agent, the environment, and the adaptation mechanism.

This perspective clarifies why the certificate is defined over a resource-aware energy that includes return and resource use. A recurring adaptation routine may improve return while using many extra updates, or may reduce computation while increasing risk. The energy

$$\mathcal{E}_{\mathrm{deploy}}(\mathcal{L}, \pi) = L_{\mathrm{lib}}(\mathcal{L}) + L_{\mathrm{ctrl}}(\pi \mid \mathcal{L}) + \mathbb{E}_{\tau \sim \bar{P}^{\pi}_{\mathcal{L}}}[C(\tau)] + \lambda[J_{\min} - J(\mathcal{L}, \pi)]_+$$

treats these quantities as jointly relevant evidence. A new operator is admitted only when the measured improvement survives the description cost of adding the operator and the uncertainty of evaluation.

The same view also explains the role of primitive unrolling. A hierarchical operator may be nonlinear and opaque, while its consequences remain assessable through the primitive adaptation trace it induces. Thus the compiler needs evidence that, when executed in the world or in a faithful evaluation process, the operator produces better resource-aware outcomes.

This is the sense in which operator growth is driven by world feedback. The environment and adaptation process supply the signals. The certificate turns those signals into an admission rule. The hierarchy grows only when repeated interaction evidence shows that a reusable adaptation operator is worth its cost.

B Related Work

Meta-reinforcement learning and in-context adaptation. Gradient-based meta-learning learns initial conditions that can adapt quickly under a prescribed update rule [6]. Latent-context methods infer task variables from experience and condition behavior on the inferred context [15]. Recurrent meta-RL, including RL^2, represents adaptation inside a recurrent state [3]. More recent sequence-model agents such as AMAGO scale in-context RL with long-context architectures [8], while LaMer studies meta-RL-induced exploration in language agents [10]. These methods learn how to adapt, with the adapted computation remaining inside a policy, hidden state, or context. Our formalism externalizes recurring adaptation procedures as callable operators whose introduction is certified.

Temporal abstraction and hierarchical RL. Options and SMDPs provide the standard mathematical foundation for temporally extended decisions [17]. Our operators inherit this foundation with an adaptation state as the underlying state. An operator may include environment actions, fast-weight updates, memory writes, and lower-rank invocations. The hierarchy is therefore over adaptation procedures, encompassing behavior in the environment together with internal adaptation steps.

Learned algorithms and test-time updates. DiscoRL shows that reinforcement-learning update rules can be discovered by meta-learning across agent populations and environments [14]. Test-time training methods update model parameters during inference, and In-Place TTT adapts a subset of LLM weights inside the deployed model [5]. These works support the broader view that learning rules and fast adaptation substrates are themselves objects of optimization. We study when repeated local adaptation traces can be consolidated into a growing hierarchy of operators.

Library learning and refactoring. LILO learns reusable symbolic abstractions by compressing and documenting code [7]. Our work is close in spirit to refactoring, with stochastic adaptation traces serving as the objects being refactored. Because our operators may be nonlinear neural kernels, we use primitive unrolling instead of symbolic denotation.

Certification and off-policy evaluation. High-confidence off-policy evaluation estimates lower bounds on policy value before deployment [18], and CoinDICE studies confidence intervals for off-policy evaluation under unknown behavior policies [2]. Confidence sequences provide time-uniform

guarantees under repeated looks and optional stopping [9]. Our certification theorem abstracts these tools into an anytime-valid lower bound on energy improvement. Replay-based instantiations still require support coverage over the hybrid adaptation decisions.

Generate–evaluate–retain systems. AlphaEvolve uses LLMs to propose algorithmic candidates and an automated evaluation loop to retain improvements [13]. This design pattern is close in spirit to our compiler, where proposal can be speculative while retention is governed by an external evaluator. The difference is the object being retained. AlphaEvolve retains programs or algorithms evaluated by task-specific metrics, whereas our framework retains closed-loop adaptation operators whose executions are unrolled into primitive adaptation traces and accepted only by a resource-aware certificate.

C Options as Degenerate Operators

This appendix makes precise the relation to the options framework. The short version is that options abstract acting, while operators abstract adapting. An option is a temporally extended environment policy. An operator is a compiled adaptation procedure over a hybrid state: it may act in the environment, update fast parameters, write memory, call lower-rank routines, and return control to its caller. Thus options are recovered as the rank-zero, no-update, no-memory projection of operators, with the statistical admission gate removed or replaced by the usual objective-driven training of option policies.

Consider a fully observed MDP

$$M = (\mathcal{S}, \mathcal{A}_{\mathrm{env}}, P, R, \gamma),$$

and a standard option

$$o = (\mathcal{I}_o, \pi_o, \beta_o), \qquad \mathcal{I}_o \subseteq \mathcal{S}, \quad \pi_o(\cdot \mid s) \in \Delta(\mathcal{A}_{\mathrm{env}}), \quad \beta_o : \mathcal{S} \to [0,1].$$

When initiated at $s_0 = s$, the option samples

$$a_t \sim \pi_o(\cdot \mid s_t), \qquad s_{t+1} \sim P(\cdot \mid s_t, a_t),$$

and then terminates according to

$$\sigma_{t+1} \sim \mathrm{Bernoulli}(\beta_o(s_{t+1})).$$

Let

$$L_o = \inf\{\ell \geq 1 : \sigma_\ell = 1\}$$

be the option duration. Classical options usually require $L_o < \infty$ almost surely. The operator definition in the main paper imposes the stronger finite-timeout condition $H_\kappa < \infty$, so the exact embedding below applies directly to bounded-duration options, or to options truncated at a fixed horizon. If one relaxes the operator timeout to a.s. finite termination, the same construction embeds proper unbounded options as well.

Define the degenerate adaptation-state embedding

$$\iota : \mathcal{S} \to \mathcal{X}, \qquad \iota(s) = (\delta_s, \varnothing, \varnothing, \varnothing).$$

Set

$$\mathcal{U}_{\mathrm{upd}} = \varnothing, \qquad \mathcal{U}_{\mathrm{mem}} = \varnothing,$$

and construct an operator

$$\kappa_o = (I_{\kappa_o}, \pi_{\kappa_o}, \beta_{\kappa_o}, H_{\kappa_o}, r_{\kappa_o}, \theta_{\kappa_o})$$

by

$$I_{\kappa_o} = \{\iota(s) : s \in \mathcal{I}_o\}, \qquad \pi_{\kappa_o}(a \mid \iota(s)) = \pi_o(a \mid s), \qquad \beta_{\kappa_o}(\iota(s)) = \beta_o(s),$$

for $a \in \mathcal{A}_{\mathrm{env}}$, with

$$r_{\kappa_o} = 0, \qquad (\kappa_o, \kappa') \notin E \quad \text{for every } \kappa'.$$

The intra-operator policy assigns zero probability to update decisions, memory decisions, and invocations. The option termination probability β_o corresponds to the operator-return probability β_{κ_o}. It should not be identified with $\mathrm{halt}_{\mathrm{adapt}}$, which terminates the whole adaptation episode rather than returning from the current operator frame.

For a discrete state space, the probability of an option trajectory

$$s_0, a_0, s_1, a_1, \ldots, a_{\ell-1}, s_\ell$$

that terminates at time ℓ is

$$\Pr_o = \left[\prod_{t=0}^{\ell-1} \pi_o(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)\right] \left[\prod_{t=1}^{\ell-1} (1 - \beta_o(s_t))\right] \beta_o(s_\ell).$$

The corresponding operator trajectory is

$$\iota(s_0), a_0, \iota(s_1), a_1, \ldots, a_{\ell-1}, \iota(s_\ell),$$

and its probability is

$$\Pr_{\kappa_o} = \left[\prod_{t=0}^{\ell-1} \pi_{\kappa_o}(a_t \mid \iota(s_t)) P(s_{t+1} \mid s_t, a_t)\right] \left[\prod_{t=1}^{\ell-1} (1 - \beta_{\kappa_o}(\iota(s_t)))\right] \beta_{\kappa_o}(\iota(s_\ell)).$$

Substituting the definitions of π_{κ_o} and β_{κ_o} gives

$$\Pr_{\kappa_o} = \Pr_o .$$

Consequently the operator's induced SMDP kernel is the pushforward of the option kernel under ι. For measurable $B \subseteq \mathcal{S}$, duration set D, and accumulated reward or cost set G,

$$K_{\kappa_o}(\iota(B), D, G \mid \iota(s)) = K_o(B, D, G \mid s).$$

The high-level Bellman equation therefore collapses from the operator SMDP equation on $\mathcal{X}$ to the usual option SMDP equation on $\mathcal{S}$:

$$V^\mu(s) = \sum_o \mu(o \mid s) \int \left[g + \gamma^\ell V^\mu(s')\right] K_o(ds', d\ell, dg \mid s).$$

The reduction also clarifies where the present framework strictly extends options. Once w_t, m_t, and ν_t are nondegenerate, an operator may represent a closed-loop adaptation computation rather than only a temporally extended environment policy. Once outgoing edges are permitted subject to $r_{\kappa'} < r_\kappa$, an operator may recursively compose lower-rank adaptation routines. Once primitive unrolling is used, heterogeneous libraries can be compared on the common trace space $\mathcal{T}_0$. Finally, once the lower-confidence admission rule is imposed, operators are not merely fitted; they are admitted only when held-out recurrence evidence exceeds operator cost, value risk, and statistical uncertainty.

D Appendix for Section 2: Adaptation Processes and SMDP Foundations

D.1 Conditional Markovization

Proposition 1 (Conditional Markovization)**.** *Under Assumption 1, the augmented state* $x_t = (\mathfrak{b}_t, w_t, m_t, \nu_t)$ *is Markov for any fixed library and fixed adaptation controller. That is, for any admissible primitive decision* u_t*,*

$$\Pr(x_{t+1} \in B \mid h_t, u_t) = \Pr(x_{t+1} \in B \mid x_t, u_t)$$

for all measurable $B \subseteq \mathcal{X}$*.*

This result is a bookkeeping device: it prevents the theory from hiding task uncertainty inside informal recurrent notation.

D.2 Measurable setup

Let $(\Omega, \mathcal{F}_\Omega)$ be the task space, $(\mathcal{O}, \mathcal{F}_\mathcal{O})$ the observation space, $(\mathcal{A}_{\text{env}}, \mathcal{F}_\mathcal{A})$ the environment action space, and $(\mathcal{X}, \mathcal{F}_\mathcal{X})$ the adaptation-state space. We assume all spaces are standard Borel spaces. For each $\omega \in \Omega$, the environment transition kernel is

$$P_\omega : \mathcal{O} \times \mathcal{A}_{\text{env}} \to \Delta(\mathcal{O}),$$

and the reward kernel is either deterministic,

$$r_t = R_\omega(o_t, a_t),$$

or stochastic with bounded support,

$$r_t \sim \mathcal{R}_\omega(\cdot \mid o_t, a_t), \qquad |r_t| \le R_{\max}.$$

The primitive update family is a set of measurable maps

$$U_\alpha : \mathcal{X} \times \mathcal{Z}_\alpha \to \mathcal{W}, \qquad \alpha \in \mathcal{U}_{\text{upd}},$$

where $\mathcal{W}$ is the fast-parameter state space. A primitive memory operator is a measurable map

$$M_\eta : \mathcal{X} \times \mathcal{Y}_\eta \to \mathcal{M}, \qquad \eta \in \mathcal{U}_{\text{mem}}.$$

The primitive cost satisfies

$$0 \le c(x, u, x') \le C_{\max}.$$

D.3 Belief update

The theoretical belief is the joint posterior

$$\mathfrak{b}_t(s,\omega) = p(s_t = s, \omega \mid h_t),$$

where

$$h_t = (o_0, u_0, r_0, \ldots, o_t, w_t, m_t, \nu_t)$$

contains all information available immediately before the next decision. For an environment action a_t, let $O_\omega(o_{t+1} \mid s_{t+1})$ be the observation likelihood and let $p_\omega(r_t \mid s_t, a_t)$ be the reward likelihood. The prediction-update equations are

$$\widetilde{\mathfrak{b}}_{t+1}(s',\omega) = \int \mathfrak{b}_t(s,\omega) P_\omega(s' \mid s, a_t) p_\omega(r_t \mid s, a_t)\, ds,$$

and

$$\mathfrak{b}_{t+1}(s',\omega) = \frac{\widetilde{\mathfrak{b}}_{t+1}(s',\omega) O_\omega(o_{t+1} \mid s')}{\int\int \widetilde{\mathfrak{b}}_{t+1}(\bar{s},\bar{\omega}) O_{\bar{\omega}}(o_{t+1} \mid \bar{s})\, d\bar{s}\, d\bar{\omega}},$$

whenever the denominator is positive. In a fully observed MDP this posterior degenerates at the observed state and leaves only the task posterior $p(\omega \mid h_t)$. If the primitive decision is an internal update or memory operation, the joint belief changes only through any observation or reward revealed during that decision.

Proof of Proposition 1. Fix a library $\mathcal{L}$, a controller π, and a primitive decision u_t. By Assumption 1, $\mathfrak{b}_t$ is the conditional law of the latent pair (s_t, ω) given the full history. Given $(\mathfrak{b}_t, u_t)$, the environment kernel, observation kernel, reward kernel, and Bayes rule above determine the conditional law of $(o_{t+1}, \mathfrak{b}_{t+1})$; no earlier part of h_t enters except through $\mathfrak{b}_t$. The fast-weight state w_{t+1} is generated from (x_t, u_t) by the selected structured update operator, the memory state m_{t+1} is generated from (x_t, u_t) by the selected memory update, and the execution state ν_{t+1} is updated from the current execution state and the selected primitive or invocation decision. Therefore the joint law of $(\mathfrak{b}_{t+1}, w_{t+1}, m_{t+1}, \nu_{t+1})$ is a function only of (x_t, u_t). Hence

$$\Pr(x_{t+1} \in B \mid h_t, u_t) = \Pr(x_{t+1} \in B \mid x_t, u_t).$$

□

D.4 Why arbitrary update vectors are excluded

Remark 1 (Structured update actions)**.** *The exclusion of arbitrary $\delta \in \mathbb{R}^d$ has substantive theoretical content. It is not merely an implementation preference; it prevents the primitive language from already containing the objects the compiler is supposed to discover.*

To see the issue, suppose the primitive action set contains every displacement of a d-dimensional fast-weight vector. Let a finite adaptation routine start at state

$$x_0 = (\mathfrak{b}_0, w_0, m_0, \nu_0)$$

and, after a sequence of environment actions, memory writes, and updates, produce terminal fast weights w_T. If arbitrary displacements are primitive, then the same terminal fast-weight component can be produced in one step by choosing

$$\delta = w_T - w_0.$$

More generally, if the primitive update is allowed to be an unrestricted measurable map from histories or states into $\mathbb{R}^d$, then the map

$$x_0 \mapsto w_T(x_0)$$

implemented by an entire closed-loop adaptation routine can be reclassified as a single primitive update. The boundary between primitive action and compiled operator disappears.

This collapse would make the hierarchy formally vacuous. Any candidate operator whose main effect is to transform fast weights could be represented by a one-step primitive update with no need for recurrence, unrolling, description cost, or admission. The compiler would no longer be discovering a reusable adaptation procedure; it would be searching directly over the full fast-weight displacement

space. In that regime, the primitive vocabulary is already as expressive as the compiled library, so compilation has no nontrivial semantic target.

The structured family

$$\delta_t = U_{\alpha_t}(x_t, \zeta_t), \qquad \alpha_t \in \mathcal{U}_{\mathrm{upd}},$$

avoids this degeneracy by making primitive updates typed, bounded, and auditable. The index α_t names an update kind, while ζ_t supplies a controlled argument such as a step size, adapter block, loss head, memory slot, rank, or residual coefficient. A compiled operator can still perform rich computations, but it must do so by composing semantically meaningful primitive decisions. This is what makes primitive traces loggable, primitive unrolling comparable across libraries, and certified recurrence a statement about reusable adaptation structure rather than a disguised search over arbitrary parameter vectors.

D.5 Finite-horizon and discounted variants

The main text uses finite primitive traces. The same construction extends to discounted infinite horizons if each operator invocation terminates almost surely and if costs and rewards are bounded. For a primitive trace $\tau = (x_0, u_0, c_0, \ldots)$, define

$$C_\gamma(\tau) = \sum_{t=0}^{\infty} \gamma^t c_t, \qquad J_\gamma(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t,$$

with $\gamma \in (0, 1)$. Boundedness gives

$$0 \le C_\gamma(\tau) \le \frac{C_{\max}}{1-\gamma}, \qquad |J_\gamma(\tau)| \le \frac{R_{\max}}{1-\gamma}.$$

D.6 SMDP induced by an operator

Let κ be a temporally extended closed-loop decision. Conditional on starting at x, executing κ produces a random sequence

$$(x_0, u_0, c_0, x_1, \ldots, x_{\tau_\kappa}), \qquad x_0 = x.$$

The induced SMDP kernel is

$$K_\kappa(B, n, G \mid x) = \Pr_\kappa\left(x_{\tau_\kappa} \in B,\, \tau_\kappa = n,\, \sum_{j=0}^{n-1} \gamma^j c_j \in G \mid x_0 = x \right).$$

For rewards, replace c_j by r_j. This is a direct specialization of the options/SMDP construction to the adaptation-state process.

Remark 2 (Timeout semantics)**.** *Throughout the paper, H_κ counts local decisions made by κ and excludes primitive time spent inside lower-rank subcalls. This is the semantics under which rank-decreasing recursion has substantive force: timeouts bound local computation, while the rank condition bounds the primitive wall-clock time created by nested subcalls.*

E Appendix for Section 3: Recursive Operators and Unrolling

E.1 Execution stack

The execution component ν_t may be represented as a bounded stack

$$\nu_t = ((\kappa^1, h^1, \ell^1), \ldots, (\kappa^q, h^q, \ell^q)),$$

where κ^i is an active operator, h^i is its internal hidden state, and ℓ^i is its local-decision count. If κ^i invokes κ', then $(\kappa', h', 0)$ is pushed onto the stack and the caller is suspended. When control returns to the top frame $(\bar\kappa, \bar h, \bar\ell)$ after one of its local decisions, execution samples an operator-return bit $\sigma' \sim \mathrm{Bernoulli}(\beta_{\bar\kappa}(x'))$ at the current adaptation state x'. If $\sigma' = 1$ or $\bar\ell = H_{\bar\kappa}$, the top frame is popped and control returns to the caller. This stack representation is optional, and it makes clear that operator calls, local-decision counts, and sampled return decisions are part of the Markov execution state.

E.2 Functional edge condition

Assumption 3 (Executable edges)**.** *For every edge $(\kappa, \kappa') \in E$, the intra-operator action space of κ contains* $\mathrm{invoke}(\kappa')$ *on states where κ' is initiated:*

$$x \in I_{\kappa'} \implies \mathrm{invoke}(\kappa') \in \mathcal{U}_\kappa(x).$$

If this condition fails, the edge is ignored by execution and is excluded from E.

This assumption rules out purely decorative DAGs. A hierarchy is functional only when higher-rank operators can actually call lower-rank operators.

E.3 Proof of well-founded execution

Proof of Theorem 1. Under the local-decision timeout semantics, define $T(r)$ as the maximum primitive wall-clock duration of any rank-r invocation. The rank condition prevents infinite pushing because along any chain of calls

$$\kappa_0 \to \kappa_1 \to \kappa_2 \to \cdots$$

the ranks satisfy

$$r_{\kappa_0} > r_{\kappa_1} > r_{\kappa_2} > \cdots .$$

Since ranks are nonnegative integers, the chain length is at most $r_{\kappa_0} + 1$. Rank-0 operators have no lower-rank operators to call, so

$$T(0) \le H_0,$$

where $H_0 = \max_{\kappa : r_\kappa = 0} H_\kappa$. Suppose $T(j) < \infty$ for all $j < r$, and let $H_r = \max_{\kappa : r_\kappa = r} H_\kappa$, which is finite because the library is finite. A rank-r operator makes at most H_r local decisions, and each local decision can trigger at most one lower-rank invocation with duration at most $\max_{j<r} T(j)$. Therefore

$$T(r) \le H_r \left(1 + \max_{j<r} T(j)\right) < \infty.$$

Induction proves finite execution for all ranks. □

E.4 Acyclicity

Corollary 1 (Rank implies acyclicity)**.** *Under Assumption 2, the directed graph (V, E) is acyclic.*

Proof. Suppose a directed cycle exists:

$$\kappa_0 \to \kappa_1 \to \cdots \to \kappa_m = \kappa_0.$$

By Assumption 2,

$$r_{\kappa_0} > r_{\kappa_1} > \cdots > r_{\kappa_m} = r_{\kappa_0},$$

a contradiction. □

E.5 Construction of the primitive unrolling kernel

For primitives $u \in \mathcal{U}_0$, define the one-step primitive unrolling as the Dirac distribution on the corresponding primitive transition. For an operator κ, define unrolling recursively. If π_κ selects a primitive decision, append the resulting primitive transition to the trace. If π_κ selects $\mathrm{invoke}(\kappa')$, sample a primitive trace from

$$U_{\mathcal{L}}(\cdot \mid x, \kappa')$$

and concatenate it with the caller's continuing execution from the returned state.

Lemma 1 (Trace concatenation preserves measurability)**.** *Let $\mathcal{T}_0$ be the disjoint union of finite primitive trace spaces*

$$\mathcal{T}_0 = \bigcup_{T=0}^{T_{\max}} \mathcal{T}_0^{(T)}$$

with the induced sigma algebra. If two finite trace kernels $Q_1(\cdot \mid x)$ and $Q_2(\cdot \mid x')$ are measurable, then their concatenation kernel is measurable.

Proof. For standard Borel spaces, finite products and finite disjoint unions are standard Borel. Concatenation is a measurable map between finite product spaces. The pushforward of a measurable product kernel under a measurable map is a measurable kernel. □

Proof of Theorem 2. Proceed by induction on rank. For rank 0, an operator can only execute primitive decisions because no lower rank exists. Finite timeout implies the generated trace is finite, and the induced distribution over $\mathcal{T}_0$ is a well-defined finite-horizon controlled-process distribution.

Assume that for all operators of rank less than r, the primitive unrolling kernel is well-defined. Consider an operator κ with rank r. During execution, κ may choose primitive decisions or invoke operators of rank less than r. Primitive decisions append primitive transitions directly. Lower-rank invocations are replaced by their already-defined unrolling kernels. By Lemma 1, concatenating these finite kernels yields a measurable finite-trace kernel. Theorem 1 guarantees that only finitely many such concatenations occur. Therefore $U_{\mathcal{L}}(\cdot \mid x, \kappa)$ is well-defined. Induction completes the proof. □

E.6 Operational semantics of unrolling

Remark 3 (Operational semantics)**.** *Primitive unrolling gives κ operational semantics through its induced primitive traces. A neural operator may produce a highly nonlinear distribution over primitive traces. The unrolling kernel only says that the operator has a well-defined operational effect in the base adaptation process. This distinction is central; the theory studies hierarchical execution and certification as operational properties.*

F Appendix for Section 4: Energy, Comparability, and Local Certificates

F.1 Controller-library ambiguity

Behavior is defined by a library together with an invocation policy or orchestration rule. When π rarely invokes a useful operator, the library may appear useless. If π is retrained after adding an operator, improvement may come from extra optimization, so the evaluation must identify the object being compared. The evaluated object is therefore one of the following pairs:

$$(\mathcal{L}, \pi),$$

or

$$(\mathcal{L}, A_{\mathrm{orch}}),$$

where A_{orch} is a specified orchestration-training rule. In the second case,

$$\pi = A_{\mathrm{orch}}(\mathcal{L}, \mathcal{D}, B),$$

where $\mathcal{D}$ is the training data and B is the permitted training budget. Comparisons must use matched budgets.

Assumption 4 (Fixed evaluation protocol)**.** *For every candidate edit $o : \mathcal{L} \mapsto o(\mathcal{L})$, the pair before the edit and the pair after the edit are produced by a pre-specified evaluation protocol. The protocol fixes whether the controller is frozen or retrained, and if retrained, fixes the data and budget available to each side.*

F.2 Description cost

A true code length requires a code. One possible construction is

$$L_{\mathrm{lib}}(\mathcal{L}) = L_{\mathrm{graph}}(V, E) + \sum_{\kappa \in V} \left[L_{\mathrm{arch}}(\kappa) + L_{\mathrm{rank}}(r_\kappa) + L_Q(\theta_\kappa) \right].$$

Here L_Q is the codelength of quantized parameters under a specified precision and prior. If the paper instead uses

$$L_{\mathrm{sur}}(\mathcal{L}) = c_V |V| + c_E |E| + c_r \sum_{\kappa \in V} r_\kappa + c_\theta \sum_{\kappa \in V} \dim(\theta_\kappa),$$

then it is a description-cost surrogate. All theorems using L_{lib} hold for either choice, provided the chosen function is fixed before certification.

F.3 Primitive-space comparability

Proposition 2 (Primitive-space comparability)**.** *Let $\mathcal{L}$ and $\mathcal{L}'$ be two libraries with possibly different operator sets. For any bounded functional $G : \mathcal{T}_0 \to [0, G_{\max}]$,*

$$\left|\mathbb{E}_{\tau\sim\bar{P}^{\pi}_{\mathcal{L}}} G(\tau) - \mathbb{E}_{\tau\sim\bar{P}^{\pi'}_{\mathcal{L}'}} G(\tau)\right| \le G_{\max}\,\mathrm{TV}\left(\bar{P}^{\pi}_{\mathcal{L}}, \bar{P}^{\pi'}_{\mathcal{L}'}\right).$$

Proof of Proposition 2. Let

$$\bar{P} = \bar{P}^{\pi}_{\mathcal{L}}, \qquad \bar{Q} = \bar{P}^{\pi'}_{\mathcal{L}'}.$$

For any bounded measurable $G : \mathcal{T}_0 \to [0, G_{\max}]$,

$$\left|\mathbb{E}_{\bar{P}} G - \mathbb{E}_{\bar{Q}} G\right| = \left|\int G(\tau)\, d(\bar{P} - \bar{Q})(\tau)\right|.$$

By the variational characterization of total variation under the convention

$$\mathrm{TV}(\bar{P}, \bar{Q}) = \sup_{A} |\bar{P}(A) - \bar{Q}(A)|,$$

we have

$$\left|\int G\, d(\bar{P} - \bar{Q})\right| \le G_{\max}\,\mathrm{TV}(\bar{P}, \bar{Q}).$$

If total variation is defined with the factor $\frac{1}{2}\|\bar{P} - \bar{Q}\|_1$, this is the same convention. □

Corollary 2 (Return comparability)**.** *If rewards are bounded by $|r_t| \le R_{\max}$ and the horizon is finite with length at most T, then for*

$$G(\tau) = \sum_{t=0}^{T-1} r_t,$$

$$|J(\mathcal{L}, \pi) - J(\mathcal{L}', \pi')| \le 2TR_{\max}\,\mathrm{TV}\left(\bar{P}^{\pi}_{\mathcal{L}}, \bar{P}^{\pi'}_{\mathcal{L}'}\right).$$

If rewards are known to be nonnegative, the factor 2 can be removed. For discounted infinite horizon with signed rewards,

$$|J_\gamma(\mathcal{L}, \pi) - J_\gamma(\mathcal{L}', \pi')| \le \frac{2R_{\max}}{1-\gamma}\,\mathrm{TV}\left(\bar{P}^{\pi}_{\mathcal{L}}, \bar{P}^{\pi'}_{\mathcal{L}'}\right).$$

F.4 Energy decomposition

Let

$$A = (\mathcal{L}, \pi), \qquad B = (o(\mathcal{L}), \pi').$$

Write

$$\mathcal{E}_{\mathrm{deploy}}(A) = L_{\mathrm{struct}}(A) + R(A) + \lambda h(J(A)),$$

where

$$L_{\mathrm{struct}}(A) = L_{\mathrm{lib}}(\mathcal{L}) + L_{\mathrm{ctrl}}(\pi \mid \mathcal{L}), \qquad R(A) = \mathbb{E}_{\tau\sim\bar{P}^{\pi}_{\mathcal{L}}}[C(\tau)], \qquad h(J) = [J_{\min} - J]_+.$$

The same definitions apply to $B = (o(\mathcal{L}), \pi')$ with its own library and controller. The true improvement is

$$\begin{aligned}\Delta\mathcal{E}_{\mathrm{deploy}} &= \mathcal{E}_{\mathrm{deploy}}(A) - \mathcal{E}_{\mathrm{deploy}}(B)\\ &= [L_{\mathrm{struct}}(A) - L_{\mathrm{struct}}(B)]\\ &\quad + [R(A) - R(B)] + \lambda[h(J(A)) - h(J(B))].\end{aligned}$$

F.5 Monotonicity of the hinge penalty

Lemma 2 (Hinge penalty is decreasing and one-Lipschitz)**.** *The function*

$$h(J) = [J_{\min} - J]_+$$

is decreasing and one-Lipschitz:

$$|h(J) - h(J')| \le |J - J'|.$$

Proof. If $J \ge J_{\min}$, then $h(J) = 0$. If $J < J_{\min}$, then $h(J) = J_{\min} - J$, which has slope -1. Thus h is decreasing with slope in $\{-1, 0\}$ wherever differentiable. Convexity and piecewise linearity give the one-Lipschitz property globally. □

F.6 Proof of certified local improvement

Proof of Theorem 3. Assume the component intervals are simultaneously valid:

$$R(A) \in [\hat{R}_A - b_R(A), \hat{R}_A + b_R(A)], \qquad R(B) \in [\hat{R}_B - b_R(B), \hat{R}_B + b_R(B)],$$

and

$$J(A) \in [\hat{J}_A - b_J(A), \hat{J}_A + b_J(A)], \qquad J(B) \in [\hat{J}_B - b_J(B), \hat{J}_B + b_J(B)].$$

Then

$$R(A) - R(B) \geq \hat{R}_A - \hat{R}_B - b_R(A) - b_R(B).$$

Since h is decreasing by Lemma 2,

$$h(J(A)) \geq h(\hat{J}_A + b_J(A)),$$

and

$$h(J(B)) \leq h(\hat{J}_B - b_J(B)).$$

Therefore

$$h(J(A)) - h(J(B)) \geq h(\hat{J}_A + b_J(A)) - h(\hat{J}_B - b_J(B)).$$

Combining the exact structural-cost difference with these two lower bounds gives

$$\Delta\mathcal{E}_{\text{deploy}} \geq \text{LCB}(\Delta\mathcal{E}_{\text{deploy}}).$$

If $\text{LCB}(\Delta\mathcal{E}_{\text{deploy}}) > 0$, then $\Delta\mathcal{E}_{\text{deploy}} > 0$. □

F.7 Simultaneous validity

Theorem 3 is conditional on simultaneous component validity. For a single edit, this can be enforced by a union bound over the four stochastic quantities $R(A), R(B), J(A), J(B)$. For adaptively chosen edits, fixed-time intervals are generally insufficient because the compiler may inspect many candidates and stop when one looks favorable. A valid sequential version requires sample splitting, fresh evaluation rollouts, or time-uniform confidence sequences.

Assumption 5 (Anytime-valid certificate)**.** *For a sequence of adaptively proposed edits $(o_t)_{t\geq 1}$ with filtration $(\mathcal{F}_t)_{t\geq 0}$, the certificate radii are chosen so that each fixed-candidate certificate is conditionally valid,*

$$\Pr(\Delta\mathcal{E}_{\text{deploy}}(o_t) \geq \text{LCB}_t(o_t) \mid \mathcal{F}_{t-1}) \geq 1 - \delta_t, \qquad \sum_{t\geq 1} \delta_t \leq \delta.$$

By a union bound, the resulting certificates satisfy

$$\Pr\left(\forall t \geq 1,\ \Delta\mathcal{E}_{\text{deploy}}(o_t) \geq \text{LCB}_t(o_t)\right) \geq 1 - \delta.$$

One concrete construction is alpha spending, which assigns nonnegative levels $(\delta_t)_{t\geq 1}$ with $\sum_t \delta_t \leq \delta$ and builds each candidate's component intervals at level δ_t using fresh certification data, sample splitting, or a confidence sequence valid under adaptive querying. A union bound then gives Assumption 5. Time-uniform confidence sequences can replace this construction when the number of validation samples per candidate is itself adaptively chosen.

Theorem 7 (All accepted edits improve energy)**.** *Under Assumption 5, the acceptance rule*

$$o_t \text{ is accepted} \quad \Longleftrightarrow \quad \text{LCB}_t(o_t) > 0$$

satisfies

$$\Pr\left(\forall t \in \mathcal{A},\ \Delta\mathcal{E}_{\text{deploy}}(o_t) > 0\right) \geq 1 - \delta,$$

where $\mathcal{A}$ is the random set of accepted edit indices.

Proof. On the event in Assumption 5, every proposed edit satisfies

$$\Delta\mathcal{E}_{\text{deploy}}(o_t) \geq \text{LCB}_t(o_t).$$

For every accepted edit, $\text{LCB}_t(o_t) > 0$, hence $\Delta\mathcal{E}_{\text{deploy}}(o_t) > 0$. The event holds with probability at least $1 - \delta$. □

F.8 Coverage caveat for replay certificates

If the certificate is computed from replay data, support is required. For a high-level policy using operators, the likelihood ratio is a Radon–Nikodym derivative over the full replay block and must include all stochastic choices that affect the primitive unrolling. In the special case where behavior-side and target-side hybrid states are both reconstructable from the logged block, it factorizes as

$$\rho(\tau) = \prod_t \frac{\pi_{\mathrm{env}}(a_t \mid x_t^\pi)\pi_{\mathrm{upd}}(\alpha_t, \zeta_t \mid x_t^\pi)\pi_{\mathrm{mem}}(\eta_t \mid x_t^\pi)\pi_{\mathrm{invoke}}(i_t \mid x_t^\pi)\pi_{\mathrm{ret}}(\sigma_t \mid x_t^\pi)}{\mu_{\mathrm{env}}(a_t \mid x_t^\mu)\mu_{\mathrm{upd}}(\alpha_t, \zeta_t \mid x_t^\mu)\mu_{\mathrm{mem}}(\eta_t \mid x_t^\mu)\mu_{\mathrm{invoke}}(i_t \mid x_t^\mu)\mu_{\mathrm{ret}}(\sigma_t \mid x_t^\mu)}.$$

If any denominator is zero where the numerator is positive, replay certification is invalid. A practical certificate must therefore include coverage diagnostics such as

$$\mathrm{ESS} = \frac{\left(\sum_i \rho_i\right)^2}{\sum_i \rho_i^2}, \qquad \max_i \rho_i \le \rho_{\max}.$$

The theory in Section 4 abstracts this issue into the validity of the LCB. Appendix G.2 gives one conservative instantiation using sample splitting and coverage-gated off-policy replay.

G Appendix for Section 5: Recurrence and Recursive Compression

G.1 Validation splits and non-circular motifs

The recurrence threshold requires that proposal and certification be separated. Let

$$\mathcal{D} = \mathcal{D}_{\mathrm{mine}} \cup \mathcal{D}_{\mathrm{fit}} \cup \mathcal{D}_{\mathrm{cert}}$$

be disjoint data splits. The mining split proposes an alignment rule $\mathcal{A}_\kappa$ and the certification split induces the held-out occurrence set $\mathcal{I}_\kappa^{\mathrm{val}}$. The fitting split trains the operator parameters θ_κ and fixes hyperparameters, stopping rules, and alignment choices. The certification split estimates savings only after the full candidate and validation protocol are frozen. If validation failures are inspected and used to revise the candidate, the next attempt is an adaptive proposal and must be covered by an anytime-valid certificate such as Assumption 5.

Assumption 6 (Split certification for motifs)**.** *For each candidate κ, the alignment rule $\mathcal{A}_\kappa$, occurrence-selection rule for $\mathcal{I}_\kappa^{\mathrm{val}}$, parameters θ_κ, hyperparameters, stopping rule, and counterfactual validation protocol are fixed before evaluating savings on $\mathcal{D}_{\mathrm{cert}}$. Any reuse of certification outcomes to modify these choices is treated as a new adaptive proposal and must be protected by simultaneous or anytime-valid coverage.*

This assumption separates discovery, tuning, and validation across data splits, avoiding certificates driven by the same idiosyncratic traces. Holding out data supports certification only when repeated validation failures are handled with sequential correction.

G.2 A full-hybrid replay certificate

We now give one concrete construction of the lower-confidence penalty used by Theorem 4. The construction is intentionally conservative. It covers the case in which the candidate, alignment rule, local replacement protocol, stopping rule, and all hyperparameters are frozen before the certification split is inspected. Adaptive mining is handled only through the split in Assumption 6 and, for repeated attempts, through alpha spending as in Assumption 5.

Fix a candidate operator κ. For each certified occurrence i, let Ω_i be an independent held-out replay block of length H, padded after adaptation halt or operator return by a null symbol. A block records the observations, rewards, and full hybrid primitive choices needed to reconstruct the behavior-side state and every target-side counterfactual state used by the fixed local protocols. In particular, it records

$$z_t = (a_t, \alpha_t, \zeta_t, \eta_t, i_t, \sigma_t),$$

including the environment action, update rule and argument, memory action, invocation choice, and operator-return bit. Let $q_{i,0}$ denote the primitive-unrolled target law of the base local protocol, and let $q_{i,1}$ denote the primitive-unrolled target law of the protocol that replaces the aligned segment by κ. The behavior law on the replay block is μ_i. Deterministic choices are represented by degenerate factors, and lower-rank invocations are expanded through their primitive unrolling kernels.

Assume full-hybrid support and bounded overlap:

$$q_{i,c} \ll \mu_i, \qquad 0 \le W_{i,c} \le \rho_{\max}$$

for $c \in \{0,1\}$, where

$$W_{i,c} = \frac{dq_{i,c}}{d\mu_i}(\Omega_i).$$

When both laws factor through common reconstruction maps $x_t^{(c)} = F_c(\Omega_{0:t-1})$ and $x_t^{\mu} = F_\mu(\Omega_{0:t-1})$, this derivative reduces to

$$W_{i,c} = \prod_{t=0}^{H-1} \frac{q_{i,c}(z_t \mid x_t^{(c)})}{\mu_i(z_t \mid x_t^{\mu})}.$$

If the support or overlap condition cannot be certified from the logging protocol and target policies, the replay certificate abstains. Let $\phi_{i,c}(\Omega_i)$ be the bounded local energy measured under protocol c, including primitive cost, call overhead, primitive unrolling cost, and any frozen continuation-value or value-risk term. Assume

$$0 \le \phi_{i,c}(\Omega_i) \le M_\Phi$$

almost surely. Define the replay saving estimator

$$\widehat{D}_i^{\mathrm{rep}} = W_{i,0}\phi_{i,0}(\Omega_i) - W_{i,1}\phi_{i,1}(\Omega_i).$$

Proposition 3 (Sample-split full-hybrid replay certificate)**.** *Under the conditions above, conditional on the frozen candidate and validation protocol,*

$$\mathbb{E}_{\mu_i}\left[\widehat{D}_i^{\mathrm{rep}}\right] = \mathbb{E}_{q_{i,0}}[\phi_{i,0}] - \mathbb{E}_{q_{i,1}}[\phi_{i,1}] = \mathbb{E}[D_i(\kappa)].$$

Moreover, with probability at least $1-\delta$*,*

$$\sum_{i=1}^{n} \mathbb{E}[D_i(\kappa)] \ge \sum_{i=1}^{n} \widehat{D}_i^{\mathrm{rep}} - 2\rho_{\max} M_\Phi \sqrt{\frac{n\log(1/\delta)}{2}}.$$

Therefore Theorem 4 applies with

$$u_n^{\mathrm{rep}}(\delta) = 2\rho_{\max} M_\Phi \sqrt{\frac{n\log(1/\delta)}{2}}.$$

For adaptively proposed candidates, assigning levels $(\delta_t)_{t\ge1}$ *with* $\sum_t \delta_t \le \delta$ *gives simultaneous validity only when each replay certificate is conditionally valid given the previous filtration, for example through fresh certification blocks, sample splitting, or a confidence sequence valid under adaptive querying.*

Proof. Because the candidate and validation protocol are fixed before the certification blocks are evaluated, the target laws $q_{i,0}, q_{i,1}$ and functionals $\phi_{i,0}, \phi_{i,1}$ are fixed conditional on the mining and fitting data. Full-hybrid support makes $W_{i,c} = dq_{i,c}/d\mu_i$ the likelihood ratio of the target primitive-unrolled block law with respect to the behavior block law. Hence the standard change-of-measure identity gives

$$\mathbb{E}_{\mu_i}[W_{i,c}\phi_{i,c}(\Omega_i)] = \mathbb{E}_{q_{i,c}}[\phi_{i,c}]$$

for $c \in \{0,1\}$. Subtracting the two identities proves unbiasedness for the frozen local saving.

The product formula above is used only when the logged block induces both behavior-side and target-side histories through the stated reconstruction maps. Otherwise the Radon–Nikodym derivative is the definition of the replay weight. The bounded-overlap and bounded-energy assumptions imply

$$\widehat{D}_i^{\mathrm{rep}} \in [-\rho_{\max} M_\Phi, \rho_{\max} M_\Phi],$$

so the range is at most $2\rho_{\max} M_\Phi$. The blocks are independent conditional on the frozen candidate, so applying the one-sided Hoeffding inequality to $\widehat{D}_i^{\mathrm{rep}}$ yields the stated lower confidence bound on the cumulative expected saving. The adaptive claim follows by applying the bound at level δ_t to each attempt only under the conditional-validity requirement in Assumption 5, then union bounding over attempts. □

This proposition is only a sufficient certificate. It does not validate ordinary replay logs that omit update choices, memory writes, invocation choices, or operator-return bits; in that case the likelihood ratio above is not defined for operators that alter those decisions. If multiple aligned occurrences from the same trajectory are used, the independent block condition should be replaced by a blocking, mixing, or martingale confidence sequence argument.

The trajectory-wise replay certificate also has an explicit horizon dependence. This is not a limitation of Theorem 4, which accepts any valid lower-confidence penalty, but it is a limitation of the concrete importance-sampling instantiation above.

Proposition 4 (Horizon dependence of trajectory-wise replay)**.** *Assume the product representation of $W_{i,c}$ holds on blocks of length H and that each per-step hybrid likelihood ratio is bounded as*

$$\frac{q_{i,c}(z_t \mid x_t^{(c)})}{\mu_i(z_t \mid x_t^{\mu})} \le \rho_1$$

whenever the ratio is defined. Then the replay penalty in Proposition 3 may be taken with

$$\rho_{\max} \le \rho_1^H, \qquad u_n^{\mathrm{rep}}(\delta) \le 2\rho_1^H M_\Phi \sqrt{\frac{n \log(1/\delta)}{2}}.$$

Consequently, if the average expected replay saving is $d > 0$ and $L_{\mathrm{add}} = 0$, the sufficient recurrence condition based on this penalty requires

$$n > \frac{2\rho_1^{2H} M_\Phi^2 \log(1/\delta)}{d^2}.$$

With $L_{\mathrm{add}} > 0$, the same exponential factor remains in the statistical term of the sufficient condition.

Proof. The product formula gives

$$W_{i,c} = \prod_{t=0}^{H-1} \frac{q_{i,c}(z_t \mid x_t^{(c)})}{\mu_i(z_t \mid x_t^{\mu})} \le \rho_1^H.$$

Substituting this bound for $\rho_{\max}$ in Proposition 3 gives the displayed confidence penalty. If $\sum_i \mathbb{E}[D_i] = nd$ and $L_{\mathrm{add}} = 0$, the recurrence threshold requires

$$nd > 2\rho_1^H M_\Phi \sqrt{\frac{n \log(1/\delta)}{2}}.$$

Dividing by $\sqrt{n}$ and squaring gives the sample-size condition. The dependence can be tight for replay blocks on which every per-step ratio equals ρ_1. □

Thus trajectory-wise replay is best understood as a conservative coverage-gated fallback. Long adaptation operators should instead be certified with a resettable generative protocol, fresh on-policy validation under the frozen candidate, or a separately proved lower-confidence penalty based on per-decision, marginal, or doubly robust estimators whenever the local energy decomposes enough to support such estimators. Those alternatives must supply their own valid $u_n(\delta)$ before they can replace $u_n^{\mathrm{rep}}(\delta)$ in Theorem 4.

G.3 Bounded savings

Let the held-out saving for occurrence i be defined by the frozen counterfactual validation protocol,

$$D_i(\kappa) = \Phi_i(\mathrm{base}) - \Phi_i(\mathrm{op}),$$

where Φ_i is a complete local energy functional for that experiment. It may include primitive cost, call overhead, a rollout-based value estimate, or a potential-based continuation value, with the decomposition specified before certification. We reserve $J^{(i)}$ for protocols that supply a canonical local value, because the paper's J is otherwise trajectory-level. Assume

$$D_i(\kappa) \in [d_{\min}, d_{\max}]$$

almost surely. Define $B = d_{\max} - d_{\min}$.

Lemma 3 (Hoeffding lower bound for cumulative saving)**.** *If $D_1, \dots, D_n$ are independent conditional on the fixed candidate κ, and $\widehat{D}_i = D_i$ are empirical savings, then with probability at least $1 - \delta$,*

$$\sum_{i=1}^{n} \mathbb{E}[D_i] \geq \sum_{i=1}^{n} D_i - B\sqrt{\frac{n \log(1/\delta)}{2}}.$$

Proof. Apply Hoeffding's inequality to the bounded variables D_i. The one-sided form gives

$$\Pr\left(\sum_{i=1}^{n} D_i - \sum_{i=1}^{n} \mathbb{E}[D_i] \geq B\sqrt{\frac{n \log(1/\delta)}{2}}\right) \leq \delta.$$

Rearranging gives the result. □

More refined bounds can replace Lemma 3. Empirical Bernstein bounds are preferable when savings have low variance, and confidence sequences are preferable when the number of checked occurrences is chosen adaptively.

G.4 Validation objective and guarantee stack

For a fixed alignment rule $\mathcal{A}_\kappa$, held-out occurrence set $\mathcal{I}_\kappa^{\mathrm{val}}$, and replacement protocol, the validation objective is

$$\mathcal{E}_{\mathrm{val}}^{\mathcal{I}_\kappa^{\mathrm{val}}}(\kappa; \mathcal{L}) = L_{\mathrm{add}}(\kappa; \mathcal{L}) - \sum_{i=1}^{n} \mathbb{E}[D_i(\kappa)].$$

Here L_{add} is the protocol-specific structural increment: it equals the library increment under frozen-controller replacement and includes the matched controller-cost increment when recurrence validation changes the controller class. All expectations are taken under the held-out counterfactual validation distribution specified before certification. The objective is normalized so that not adding the operator has value 0. Therefore the recurrence certificate proves a strictly local statement: accepted evidence implies $\mathcal{E}_{\mathrm{val}}^{\mathcal{I}_\kappa^{\mathrm{val}}}(\kappa; \mathcal{L}) < 0$ with the stated confidence.

This objective is distinct from execution validity and from deployment improvement. Execution validity is semantic: rank-decreasing calls and finite local-decision timeouts imply finite primitive unrolling. Frozen validation improvement is statistical: the lower-confidence recurrence bound shows that repeated isolated savings exceed description cost under the fixed protocol. Deployment improvement is a transfer claim: it follows only under regeneration, coupling bounds such as Theorem 5, or a direct deployment certificate. This separation is what the emergence sandwich summarizes.

G.5 Proof of recurrence threshold

Proof of Theorem 4. Let

$$\bar{S}_n(\kappa) = \sum_{i=1}^{n} \mathbb{E}[D_i(\kappa)]$$

be the expected cumulative saving under the frozen validation protocol. By assumption, with probability at least $1 - \delta$,

$$\bar{S}_n(\kappa) \geq \sum_{i=1}^{n} \widehat{D}_i(\kappa) - u_n(\delta).$$

If

$$\sum_{i=1}^{n} \widehat{D}_i(\kappa) > L_{\mathrm{add}}(\kappa; \mathcal{L}) + u_n(\delta),$$

then

$$\bar{S}_n(\kappa) > L_{\mathrm{add}}(\kappa; \mathcal{L}).$$

Thus the expected cumulative isolated saving exceeds the protocol-specific structural increment, which is exactly $\mathcal{E}_{\mathrm{val}}^{\mathcal{I}_\kappa^{\mathrm{val}}}(\kappa; \mathcal{L}) < 0$. A claim about full closed-loop trajectory energy requires the separate deployment certificate of Theorem 3 or the coupling conditions of Theorem 5. □

G.6 Proof of local-to-global deployment lower bound

Proof of Theorem 5. Start from the assumed regenerative decomposition of $\Delta\mathcal{E}_{\mathrm{deploy}}$. The non-overlap or deterministic-schedule condition makes the segment savings marginal with respect to the same replacement plan, so the sum $\sum_i \mathbb{E}[D_i^{\mathrm{seg}}]$ does not count any primitive transition twice. For each scheduled occurrence, the only remaining downstream term is the difference between the base and operator terminal-state laws evaluated by the same continuation-energy function G_i. Since $|G_i| \le V_{\max}$, for any two terminal-state laws P and Q,

$$|\mathbb{E}_P[G_i] - \mathbb{E}_Q[G_i]| \le 2V_{\max}\,\mathrm{TV}(P, Q).$$

Applying this inequality with $P = P^{\kappa}_{\mathrm{term},i}$ and $Q = P^{\mathrm{base}}_{\mathrm{term},i}$ gives a worst-case loss of at most $2V_{\max}\epsilon_i$ for occurrence i. Substituting these bounds into the regenerative decomposition proves the stated inequality. If the terminal distributions match exactly, the scheduled continuation terms cancel and the expected segment-only savings add. If the local validation functional already includes an approximate continuation value $\widehat{V}$, this proof applies only after replacing the TV term by an explicit residual bound on $G_i - \widehat{V}$. □

G.7 Directional continuation certificates

The TV term in Theorem 5 is deliberately worst case: it protects against arbitrary downstream damage after the replacement changes the terminal adaptation state. This can be vacuous for the state-changing operators of interest. For example, if a deterministic base segment terminates at x and the operator terminates at a different augmented state y, then

$$P^{\mathrm{base}}_{\mathrm{term},i} = \delta_x, \qquad P^{\kappa}_{\mathrm{term},i} = \delta_y, \qquad \mathrm{TV}(\delta_x, \delta_y) = 1.$$

Thus a reset of fast weights, a memory write, or a belief update can incur the full $2V_{\max}$ penalty even when the change is beneficial for continuation. More generally, if the terminal laws differ by a constant TV amount over n scheduled replacements, the symmetric penalty scales as $\Theta(n)$ and can dominate the segment savings.

A sharper deployment bridge should certify the signed continuation drift rather than the magnitude of the terminal-state distribution shift. The following variant uses a continuation surrogate and one-sided residuals.

Proposition 5 (Directional local-to-global lower bound)**.** *Assume the same frozen-controller replacement schedule and regenerative decomposition as Theorem 5. For each occurrence i, let $\widehat{G}_i$ be a measurable continuation surrogate. Suppose there are nonnegative residuals η_i^{κ} and η_i^{base} such that*

$$\mathbb{E}_{x\sim P^{\kappa}_{\mathrm{term},i}}\left[G_i(x) - \widehat{G}_i(x)\right] \le \eta_i^{\kappa}, \qquad \mathbb{E}_{x\sim P^{\mathrm{base}}_{\mathrm{term},i}}\left[\widehat{G}_i(x) - G_i(x)\right] \le \eta_i^{\mathrm{base}}.$$

Then

$$\Delta\mathcal{E}_{\mathrm{deploy}} \ge \bar{S}_n^{\mathrm{seg}}(\kappa) - L_{\mathrm{add}}(\kappa; \mathcal{L}) - \sum_{i=1}^{n}\left[\mathbb{E}_{x\sim P^{\kappa}_{\mathrm{term},i}}\widehat{G}_i(x) - \mathbb{E}_{x\sim P^{\mathrm{base}}_{\mathrm{term},i}}\widehat{G}_i(x)\right] - \sum_{i=1}^{n}\left(\eta_i^{\kappa} + \eta_i^{\mathrm{base}}\right).$$

Proof. For a scheduled occurrence, the one-sided residual assumptions imply

$$\mathbb{E}_{P^{\kappa}_{\mathrm{term},i}} G_i - \mathbb{E}_{P^{\mathrm{base}}_{\mathrm{term},i}} G_i \le \mathbb{E}_{P^{\kappa}_{\mathrm{term},i}}\widehat{G}_i - \mathbb{E}_{P^{\mathrm{base}}_{\mathrm{term},i}}\widehat{G}_i + \eta_i^{\kappa} + \eta_i^{\mathrm{base}}.$$

Substituting this upper bound on the continuation drift into the regenerative decomposition of Theorem 5 gives the claim. □

Unlike the symmetric TV bridge, Proposition 5 preserves the sign of the continuation change. If the operator moves terminal states toward lower continuation energy, the bracketed surrogate drift is negative and strengthens the deployment lower bound. If the local validation protocol estimates this drift from data, the empirical estimate must be replaced by an upper confidence bound and the residuals must cover the approximation and statistical error. A pure divergence penalty can be used as part of such a residual, but it does not by itself solve the problem unless it is paired with a signed continuation surrogate or an equivalent one-sided coupling.

G.8 Proof of emergence sandwich

Proof of Theorem 6. Fix the event on which all recurrence lower-confidence bounds used up to time T are valid. The assumed confidence allocation gives this event probability at least $1-\delta$. On this event, any accepted edit satisfies the displayed recurrence threshold, so Theorem 4 gives

$$\Delta\mathcal{E}^{\mathrm{loc}}_{\mathrm{val},t} = \sum_{i=1}^{n_t} \mathbb{E}[D_{t,i}] - L_{\mathrm{add}}(\kappa_t;\mathcal{L}_t) > 0.$$

The rank check and primitive-unrolling check ensure well-founded execution by Theorem 1 and Theorem 2. Thus every accepted edit has the stated execution and local-validation guarantees.

If the coupling assumptions of Theorem 5 also hold at step t, apply that theorem to the segment-only savings $D^{\mathrm{seg}}_{t,i}$. This yields

$$\Delta\mathcal{E}_{\mathrm{deploy},t} \geq \sum_{i=1}^{n_t} \mathbb{E}[D^{\mathrm{seg}}_{t,i}] - L_{\mathrm{add}}(\kappa_t;\mathcal{L}_t) - 2V_{\max}\sum_{i=1}^{n_t}\epsilon_{t,i} = \Delta\mathcal{E}^{\mathrm{seg}}_{\mathrm{val},t} - 2V_{\max}\sum_{i=1}^{n_t}\epsilon_{t,i}.$$

The deployment layer therefore uses only segment-only validation gain; any augmented local value term remains confined to $\Delta\mathcal{E}^{\mathrm{loc}}_{\mathrm{val},t}$ unless a separate residual approximation bound is supplied. □

G.9 No-free-hierarchy corollary

Corollary 3 (No-free-hierarchy)**.** *If every proposed operator κ satisfies*

$$\sum_{i=1}^{n} \widehat{D}_i(\kappa) \leq L_{\mathrm{add}}(\kappa;\mathcal{L}) + u_n(\delta),$$

then no operator is certified by the recurrence rule.

Proof of Corollary 3. The acceptance rule in Theorem 4 requires

$$\sum_{i=1}^{n} \widehat{D}_i(\kappa) > L_{\mathrm{add}}(\kappa;\mathcal{L}) + u_n(\delta).$$

If every proposed κ violates this strict inequality, no proposed operator passes the recurrence certificate. □

G.10 Recursive compression

Let $\mathcal{L}_{<r}$ denote a library containing only operators of rank below r. A rank-r candidate κ may invoke operators in $\mathcal{L}_{<r}$. Its high-level execution induces a primitive unrolling distribution by Theorem 2. Held-out savings are computed by the frozen counterfactual validation protocol after sampling or evaluating this unrolled primitive trace.

Lemma 4 (Recursive candidates reduce to primitive candidates)**.** *Under Assumption 2, any rank-r candidate κ induces a finite primitive unrolling kernel. Therefore any fixed counterfactual validation functional for κ can be evaluated on primitive traces.*

Proof. This is a direct application of Theorem 2. Rank-decreasing calls and finite local-decision timeouts imply that every high-level execution expands into a finite primitive trace distribution. □

Corollary 4 (Recursive compression condition)**.** *Let κ be a candidate rank-r operator whose executable calls satisfy*

$$(\kappa,\kappa') \in E \implies r_{\kappa'} < r.$$

Assume its held-out savings are computed after primitive unrolling. If the recurrence threshold in Theorem 4 holds for κ, then adding κ gives $\mathcal{E}^{\mathcal{I}^{\mathrm{val}}_{\kappa}}_{\mathrm{val}}(\kappa;\mathcal{L}) < 0$ while preserving well-founded execution.

Proof of Corollary 4. By the rank condition, adding κ preserves well-founded execution by Theorem 1. By Lemma 4, the candidate can be evaluated on primitive traces after unrolling. The recurrence threshold then applies to the fixed counterfactual validation objective. Hence the edit passes the recurrence certificate for that objective and preserves finite execution. □

G.11 Operator cost amortization

The recurrence threshold has the following amortization form:

$$\frac{L_{\mathrm{add}}(\kappa;\mathcal{L})+u_n(\delta)}{n} < \frac{1}{n}\sum_{i=1}^{n}\widehat{D}_i(\kappa).$$

The left side is the per-use burden of adding the operator. The right side is observed per-use saving. This form clarifies that rare motifs require very large savings before compilation.

H Appendix for Section 6: Proposal Support and Revival

H.1 Proof of type-level non-extinction

Proposition 6 (Type-level non-extinction)**.** *If $G_0(r) > 0$ and $\rho_t \geq \rho_{\min} > 0$, then*

$$q_t(r \mid c_t) \geq \rho_{\min} G_0(r)$$

for every compiler step t.

Proof of Proposition 6. By definition,

$$q_t(r \mid c_t) = (1-\rho_t)\tilde{q}_t(r \mid c_t) + \rho_t G_0(r).$$

Since $\tilde{q}_t(r \mid c_t) \geq 0$ and $\rho_t \geq \rho_{\min}$,

$$q_t(r \mid c_t) \geq \rho_t G_0(r) \geq \rho_{\min} G_0(r).$$

□

H.2 Proposal hitting bound

Theorem 8 (Proposal hitting bound)**.** *Assume that at every compiler step, $q_t(r^\star \mid c_t) \geq \rho_{\min} G_0(r^\star)$, that $Q_t(o \in \mathcal{G} \mid r^\star, c_t) \geq p_\star$, and that candidates in $\mathcal{G}$ are accepted with probability at least $a_\star$. Then the expected time to propose and accept an edit from $\mathcal{G}$ is at most $\frac{1}{\rho_{\min} G_0(r^\star) p_\star a_\star}$.*

Proof of Theorem 8. At each compiler step, the probability of drawing proposal type $r^\star$ is at least

$$\rho_{\min} G_0(r^\star).$$

Conditional on drawing this type, the probability of landing in $\mathcal{G}$ is at least $p_\star$. Conditional on landing in $\mathcal{G}$, the probability of acceptance is at least $a_\star$. Therefore each step succeeds with probability at least

$$p_{\mathrm{hit}} = \rho_{\min} G_0(r^\star) p_\star a_\star.$$

The hitting time is stochastically dominated by a geometric random variable with success probability p_{hit}. Its expectation is at most

$$\frac{1}{p_{\mathrm{hit}}} = \frac{1}{\rho_{\min} G_0(r^\star) p_\star a_\star}.$$

□

H.3 Finite-cover discovery bound

The hitting theorem can be strengthened when the proposal floor ranges over a finite cover of operator neighborhoods. It remains a discovery statement under the assumption that one cover element has a stable positive validation margin.

Let $\mathcal{K}$ be an admissible candidate class equipped with a primitive-unrolling pseudometric d_{op}. Let $\{\bar{\kappa}_1, \ldots, \bar{\kappa}_M\}$ be an ϵ-cover. Fix a certification batch size N and suppose there exists an index $j^\star$ such that every candidate in the cell of $\bar{\kappa}_{j^\star}$ has per-occurrence expected saving exceeding its amortized one-time cost by at least $m > 0$:

$$\bar{\mu}_N(\kappa) := \mathbb{E}[D(\kappa)] - \frac{L_{\mathrm{add}}(\kappa;\mathcal{L})}{N} \geq m.$$

Equivalently, the expected cumulative recurrence surplus $N\mathbb{E}[D(\kappa)] - L_{\mathrm{add}}(\kappa;\mathcal{L})$ is at least Nm. Assume each empirical saving sample is bounded in an interval of width B, the proposal process samples each cover element with probability at least p_{cell}, and certification of a tested candidate uses fresh samples independent of proposal and previous tests. For a uniform floor over cells, $p_{\mathrm{cell}} = 1/M$; if the cells are reached only through the revival mixture in Section 6, typically $p_{\mathrm{cell}} \geq \rho_{\min}/M$.

Theorem 9 (Finite-cover discovery)**.** *Under the assumptions above, let*

$$T \geq \frac{1}{p_{\text{cell}}} \log \frac{2}{\delta}$$

and test each proposed cover element using the fixed batch size N, where

$$N \geq \frac{2B^2}{m^2} \log \frac{2T}{\delta}.$$

Using fresh validation samples, accept only when the empirical cumulative surplus satisfies

$$\sum_{i=1}^{N} \widehat{D}_i(\kappa) - L_{\text{add}}(\kappa; \mathcal{L}) > Nm/2.$$

Then with probability at least $1 - \delta$, the compiler proposes and certifies a candidate from the useful cover cell within T attempts.

Proof. The probability of missing the useful cell in T independent floor attempts is at most

$$(1 - p_{\text{cell}})^T \leq \exp(-p_{\text{cell}} T) \leq \delta/2.$$

Condition on the event that the useful cell is sampled. For any tested candidate in that cell, the expected cumulative surplus

$$\mu_N(\kappa) = \sum_{i=1}^{N} \mathbb{E}[D_i(\kappa)] - L_{\text{add}}(\kappa; \mathcal{L})$$

is at least Nm. Since L_{add} is deterministic and paid once per operator, concentration applies only to the cumulative saving term. Hoeffding's inequality gives

$$\Pr\left(\sum_{i=1}^{N} \widehat{D}_i(\kappa) - L_{\text{add}}(\kappa; \mathcal{L}) \leq Nm/2 \right) \leq \exp\left(-\frac{Nm^2}{2B^2} \right) \leq \frac{\delta}{2T}.$$

A union bound over at most T tested candidates shows that every useful tested candidate is retained with probability at least $1 - \delta/2$. Combining the proposal and validation failures proves the claim. □

The theorem explains the scaling under strong assumptions. The one-time operator cost is charged once to the cumulative batch surplus rather than subtracted from each occurrence. The cover must be finite at the chosen resolution, the useful margin must be stable across an entire cover cell, and each test must use fresh or anytime-corrected validation data. Substituting $p_{\text{cell}} = 1/M$ recovers $T \geq M \log(2/\delta)$, while a revival-mixture floor with $p_{\text{cell}} \geq \rho_{\min}/M$ gives $T \geq (M/\rho_{\min}) \log(2/\delta)$. For continuous neural operators this is best read as a neighborhood-discovery statement over useful operator regions.

H.4 Exact cache revival

Let $\mathcal{H}_t$ be a dormant cache. Each element is

$$(\kappa, \theta_\kappa, E_\kappa, \text{meta}_\kappa),$$

where E_κ stores admissible call edges and meta_κ stores evidence used by the previous certificate.

Proposition 7 (Cached reactivation support)**.** *Suppose a dormant operator $\kappa \in \mathcal{H}_t$ is sampled for reactivation with probability at least $p_{\text{cache}} > 0$, and its acceptance probability under the current validation distribution is at least $a_{\text{cache}} > 0$. Then the expected time to reactivate κ is at most*

$$\frac{1}{p_{\text{cache}} a_{\text{cache}}}.$$

Proof. Each compiler step reactivates κ with probability at least $p_{\text{cache}} a_{\text{cache}}$. The same geometric domination argument as in Theorem 8 applies. □

Remark 4 (Rediscovery versus reactivation)**.** *A cache gives exact reactivation positive probability by placing an atom at stored parameters. Under an absolutely continuous parameter proposal, meaningful rediscovery is instead defined by a positive-measure equivalence class, such as a neighborhood of operators with positive certified improvement.*

H.5 Proposal Regret and Discovery Support

A proposal scheduler may use an expert mixture

$$Q_t = \sum_{j=1}^{M} w_{t,j} Q_{t,j}.$$

Online learning can make the scheduler competitive with the best proposal expert in hindsight. Discovery follows only when at least one expert assigns nonzero probability to useful operator regions.

Proposition 8 (Regret-to-discovery separation)**.** *Suppose every proposal expert $Q_{t,j}$ assigns zero probability to a useful region $\mathcal{G}$ at every step. Then any mixture over these experts also assigns zero probability to $\mathcal{G}$, regardless of its regret guarantee.*

Proof. For every j and t,

$$Q_{t,j}(\mathcal{G}) = 0.$$

Therefore

$$Q_t(\mathcal{G}) = \sum_{j=1}^{M} w_{t,j} Q_{t,j}(\mathcal{G}) = 0.$$

Thus proposals remain outside $\mathcal{G}$. □

I Appendix for Section 7: Extensions and Technical Caveats

I.1 Continuous operator neighborhoods

For nonlinear neural operators, exact equality of parameters is rarely the right equivalence relation. Let d_{op} be a pseudometric on operator kernels, for example

$$d_{\mathrm{op}}(\kappa, \kappa') = \sup_{x \in \mathcal{X}_0} \mathrm{TV}\left(U_{\mathcal{L}}(\cdot \mid x, \kappa), U_{\mathcal{L}}(\cdot \mid x, \kappa')\right)$$

on a validation state set $\mathcal{X}_0$. A useful neighborhood of $\kappa^\star$ can be defined as

$$\mathcal{N}_\epsilon(\kappa^\star) = \{\kappa : d_{\mathrm{op}}(\kappa, \kappa^\star) \le \epsilon\}.$$

Discovery claims for continuous operators should concern such neighborhoods as the meaningful units of rediscovery.

I.2 Matched controller budgets

When the controller is retrained after adding an operator, a fair comparison uses matched budgets defined by

$$\pi_A = A_{\mathrm{orch}}(\mathcal{L}, \mathcal{D}, B), \qquad \pi_B = A_{\mathrm{orch}}(o(\mathcal{L}), \mathcal{D}, B),$$

where B is the same update, data, and compute budget. This constraint separates operator gain from controller-training gain.

I.3 Replay certification for hybrid traces

For replay-based certification, the behavior policy must cover all stochastic choices in the candidate's unrolled execution. A sufficient support condition is

$$\mu(z_t \mid x_t) > 0 \quad \text{whenever} \quad \pi(z_t \mid x_t) > 0,$$

where

$$z_t = (a_t, \alpha_t, \zeta_t, \eta_t, i_t, \sigma_t)$$

contains environment action, update choice and argument, memory choice, invocation choice, and operator-return choice. A replay buffer that logs only (o_t, a_t, r_t, o_{t+1}) supports certification only for adaptation operators whose updates, invocations, and return decisions remain covered by the behavior process.

I.4 Relation between local and global objectives

The acceptance theorem guarantees the following local improvement

$$\mathcal{E}_{\mathrm{deploy}}(\mathcal{L}_{t+1}, \pi_{t+1}) < \mathcal{E}_{\mathrm{deploy}}(\mathcal{L}_t, \pi_t)$$

on the certificate event. Global optimality requires additional conditions. Global statements require compactness of the library class, lower semicontinuity of the energy, sufficient proposal coverage, and a rule for escaping local minima. These assumptions exceed the scope of the present paper and remain outside the main claim.